Computers & Security 159 (2025) 104672

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full Length Article

The hidden complexities of Android TPL detection: An empirical analysis of techniques, challenges, and effectiveness

Lige Zhan [a], Jiang Ming [b], Jianming Fu [a,*], Guojun Peng [a], Letian Sha [c], Lili Lan [a]

[a] School of Cyber Science and Engineering, Wuhan University, No. 340, Luoyu Road, Wuhan, 430072, Hubei, China
[b] Department of Computer Science, Tulane University, 6823 St. Charles Avenue, New Orleans, 70118, LA, United States of America
[c] School of Computer Science, Nanjing University of Posts and Telecommunications, No. 9, Wenyuan Road, Nanjing, 210023, Jiangsu, China

ARTICLE INFO

Keywords:
Software supply chain security
Third-party library detection
Code obfuscation and optimization
Vulnerability identification

ABSTRACT

Third-party libraries (TPLs) play a crucial role in Android application (app) development and have become an indispensable part of the Android ecosystem. However, TPLs also introduce potential security risks, as they may propagate 1-day vulnerabilities or even malicious code into apps. Moreover, certain downstream tasks, such as app clone detection, license violation identification and patch presence test, require accurate TPL detection as a prerequisite. Consequently, TPL detection has gained increasing importance over the past decade in improving maintainability and enhancing security within the software supply chain. To ensure robustness against external factors and precise vulnerability identification, modern library detection tools, in addition to recognizing TPL variety, must be resilient to code obfuscation and optimization, and must also be capable of accurately identifying library versions. Although recent studies have reported progress in addressing these issues, none have conducted a comprehensive evaluation to determine whether the proposed methods effectively overcome these challenges. Furthermore, critical aspects such as tool performance on real-world apps, as well as the generalizability of existing approaches, are frequently overlooked in current research.

To gain deeper insights into TPL detection research, we conducted a comprehensive empirical analysis of state-of-the-art approaches in this domain. This study begins by summarizing the common technologies used at each stage of the TPL detection process, followed by an analysis of the prevalence of code obfuscation and optimization in real-world apps to identify key external factors that hinder effective library detection. Next, we evaluate the performance of cutting-edge tools on multiple ground-truth datasets to validate our findings. Specifically, we systematically analyze the methodologies employed by these tools, assessing their capabilities in TPL variety detection, version identification, resilience to common obfuscation and optimization techniques, and the underlying causes of their failures. Finally, we assessed the generalizability of these tools by comparing their performance across diverse datasets and validating them with real-world data. Our findings confirm that obfuscation and optimization are indeed prevalent in real-world scenarios. However, the code transformations introduced by these techniques often exceed the scope of scenarios considered in prior TPL detection studies. We also observe that even the most advanced detection features struggle to accurately differentiate between library versions. In addition to errors caused by obfuscation and optimization, overly simplistic library features can further contribute to false positives. Moreover, while most tools perform well on their own curated datasets and show reduced performance on external datasets, their effectiveness in real-world scenarios does not exhibit a substantial disparity. Overall, this paper presents a comprehensive analysis and evaluation of current TPL detection techniques, providing a solid foundation for future research in this area.

1. Introduction

Android currently holds a dominant 71.6% market share globally (Statcounter, 2025), a success largely attributed to the convenience offered by its robust software supply chain, particularly its extensive array of third-party libraries (TPLs). TPLs play a critical role in Android application (app) development by shortening development cycles and providing essential functionalities such as advertising, communication, social media, and e-commerce (Wu et al., 2023). Existing studies (Liu et al., 2015) have shown that approximately 57% of apps

* Corresponding author.
E-mail address: jmfu@whu.edu.cn (J. Fu).

https://doi.org/10.1016/j.cose.2025.104672
Received 9 March 2025; Received in revised form 30 August 2025; Accepted 13 September 2025
Available online 1 October 2025
0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

incorporate TPLs, with nearly 60% of Android code derived from those libraries (Wang and Guo, 2017). Additionally, a single app can integrate dozens of TPLs (Zhan et al., 2021b; Ma et al., 2016). However, the pervasive use of TPLs also introduces significant security risks (Chen et al., 2016; Yasumatsu et al., 2019; Zhao et al., 2023b).

As key upstream components in the software supply chain, security vulnerabilities in TPLs can spread throughout the supply chain, affecting all downstream entities (Li et al., 2017a; Tang et al., 2019a; Chen et al., 2018, 2019; Watanabe et al., 2017). A prominent example is CVE-2020-8840 (NVD, 2025), a critical vulnerability in *Jackson-databind*, one of the most widely used JSON processing libraries. This supply chain vulnerability has been assigned a critical CVSS score of 9.8 and enables attackers to perform arbitrary remote code execution. Moreover, even libraries provided by major technology companies, such as the Facebook SDK and Baidu SDK, have been found to contain severe security vulnerabilities, which have put millions of smartphone users' accounts at risk (The Hacker News, 2025a,b). These security threats highlight the critical need for TPL detection, which identifies different TPL variety and versions to uncover potential vulnerabilities. Additionally, tasks such as repackaged app inspection (Zhou et al., 2013; Wang et al., 2015; Zhan et al., 2019), library isolation (Liu et al., 2015), patch updates (Duan et al., 2019; Derr et al., 2017; Huang et al., 2019), and patch presence test (Zhang and Qian, 2018; Dai et al., 2020; Jiang et al., 2020; Xie et al., 2023; Xu et al., 2020; Pan et al., 2024; Zhan et al., 2024a,b) depend on accurate TPL detection. Consequently, research on TPL detection for the Android platform has gained significant attention in recent years (Zhan et al., 2021c).

In general, the study of TPL detection began with clustering-based approaches (Li et al., 2017b). By analyzing features from thousands of apps, the clustering algorithm can capture characteristics from commonly used TPLs. However, the widespread adoption of Android obfuscators (Guardsquare, 2025; Allatori, 2025; PreEmptive, 2025; Claudiu-Georgiu, 2025) presents significant challenges to this approach. While obfuscation preserves semantics, it substantially modifies the syntactic representation of features, thereby hindering clustering algorithms from obtaining optimal parameters. Notably, over 50% of popular apps have undergone obfuscation (Wermke et al., 2018). In response, current research primarily employs similarity-comparison based approaches (Backes et al., 2016; Zhang et al., 2019; Zhan et al., 2021a; Huang et al., 2022; Xie et al., 2024). This approach detects potential TPLs by comparing customized features extracted from both the target app and collected libraries. These customized features are designed to enhance resilience against obfuscation and facilitate fine-grained detection of TPL versions. Version identification is particularly crucial as it helps uncover hidden 1-day vulnerabilities in apps, making it a primary research objective. As understanding of obfuscation deepens, researchers have delineated the core set of obfuscation techniques that TPL detection tools must address. Since ATVHunter (Zhan et al., 2021a), TPL detection tools have aimed to resist all techniques within this defined scope. Furthermore, the rapid evolution of the Android ecosystem – particularly in compiler optimizations (e.g., R8 optimization) – poses new challenges for TPL detection. Consequently, state-of-the-art tools must also mitigate the impact of such code optimizations (Xie et al., 2024).

Although research on TPL detection is expanding, and most studies consistently aim to address challenges posed by obfuscation and optimization, the real-world prevalence of optimized apps, particularly when obfuscation is also taken into account, remains unclear. This gap in research makes it difficult to determine whether anti-obfuscation and anti-optimization strategies are truly necessary. Moreover, existing work does not provide a comprehensive discussion of the strengths and weaknesses of the methodologies employed at each step of the detection process. Most studies evaluate tool performance on limited datasets, resulting in an incomplete and potentially biased understanding of their effectiveness, efficiency, and generalizability. In particular, there is a lack of systematic analysis of the extent to which obfuscation and optimization impact these techniques, as well as the common-used methods' capabilities for version identification. The underlying causes of detection errors in existing methods and the generalizability of these tools remain insufficiently explored. Furthermore, real-world evaluations of state-of-the-art tools are still lacking.

To address these challenges, this paper presents an empirical study on existing TPL detection approaches. We begin by analyzing the prevalence of obfuscation and optimization in large-scale real-world apps to identify key external factors that hinder TPL detection. Next, this study conducts a comprehensive evaluation of state-of-the-art TPL detection tools, assessing their effectiveness across diverse datasets rather than limiting the analysis to the datasets provided by the tools themselves. We not only perform an in-depth analysis of three key capabilities in TPL detection – including TPL variety and version identification, and resilience against obfuscation and optimization – but also analyze the primary causes of false positives and false negatives. In the efficiency evaluation, we present both preprocessing time and matching time to better reflect the tools' overall efficiency. Subsequently, this study evaluates the generalizability of the tools by comparing their performance across multiple datasets and contrasting these results with those reported in their original publications. To better demonstrate the practical performance of these tools, we also conduct real-world experiments. Finally, we attempt to construct a new tool by leveraging the strengths of existing techniques, and summarize the lessons learned and insights gained throughout our study. Through the comprehensive analysis and evaluation, we aim to enhance understanding of current TPL detection approaches and provide statistically grounded support for future research in this field. In summary, the contributions of this paper are as follows:

- We review existing TPL detection research and decompose the employed techniques according to the stages of the detection pipeline. For each stage, we summarize the strengths and limitations of current methods and highlight the key challenges that remain.
- We construct a large-scale dataset comprising original, obfuscated, optimized, and real-world app variants to evaluate the performance of cutting-edge tools across diverse app types. Utilizing this dataset, we perform comprehensive evaluations to assess these tools with respect to their effectiveness, efficiency, and generalizability.
- We are the first to investigate the extent to which existing methodologies can withstand code obfuscation and optimization, as well as whether commonly used TPL detection features employed by current approaches can effectively distinguish different library versions.
- Based on our findings, we summarize the lessons learned and offer research recommendations to inform future advancements in TPL detection.

The remainder of this paper is organized as follows: Section 2 introduces the background of TPL detection. Section 3 describes how we selected the target papers and briefly summarizes them in chronological order. Section 4 presents the methodologies adopted by existing TPL approaches at different stages. In Section 5, we conduct an empirical study using multiple datasets to evaluate the effectiveness, efficiency, and generalizability of existing work. Section 6 presents our attempt to design a new approach by integrating existing techniques. Section 7 discusses the lessons learned and our insights for future research. Finally, Section 8 concludes the paper.

2. Background

In this section, we provide background information on TPL detection, including its importance in software supply chain security, the existing methodologies and main detection processes, and how obfuscation and optimization may hinder detection accuracy, along with the related technologies.

2.1. TPL detection in supply chain security

The software supply chain encompasses libraries, components, tools, and other critical elements used in software development. As a core component, Third-party libraries (TPLs) provide open-source, reusable code modules that enable developers to efficiently implement specific functionalities while avoiding redundant code construction. These TPLs offer a wide variety of development kits, functional modules, and rich user interface components. In Android applications (apps), TPLs typically fall into two categories: Java libraries and native libraries (C/C++ code). Java libraries are typically packaged as *JAR* (Java Archive) or *AAR* (Android Archive) files, containing compiled bytecode and associated resources that can be directly integrated into an app through standard dependency management systems such as Maven (Apache, 2025) or Gradle (2025). Developers can locate specific TPLs by using a combination of group ID, artifact ID, and version number. These libraries execute within the Java Virtual Machine (JVM) or Android Runtime (ART/Dalvik) environments without requiring additional interfacing mechanisms. In contrast, native libraries are compiled into platform-specific machine code and stored as *SO* (Shared Object) files. To interact with Java-based applications, native libraries typically need to adhere to the Java Native Interface (JNI) specification, which defines a standardized framework for invoking native functions from Java and for calling Java methods from native code.

Although TPLs greatly facilitate development, they also pose considerable security risks. Situated at the upstream echelon of the supply chain Ntousakis et al. (2021), security risks inherent in TPLs may pose a significant threat to the stability and continuity of the downstream dependencies (Viertel et al., 2019). The oversight of security vulnerabilities within TPLs can lead to severe repercussions, including data privacy breaches, financial damages, and unauthorized remote access to user devices (Jarukitpipat et al., 2023). Furthermore, certain studies (Zhou et al., 2013; Liu et al., 2015; Wang et al., 2015; Zhan et al., 2019) primarily focus on analyzing the core code of apps, as their main objective is to detect functional code reuse within the app itself rather than in TPLs. As a result, the inclusion of library code introduces considerable noise, which can undermine the accuracy and reliability of their analyses. For example, Wukong (Wang et al., 2015) identifies clone apps by comparing the code features of an original app with those of the target. However, if two apps are not clone pairs but share common TPLs, their similarity may increase, potentially resulting in false positives. Nowadays, the increasingly severe security risks associated with TPLs have become a serious threat to the software supply chain ecosystem, thereby spurring significant research efforts in TPL detection (Xu and Yuan, 2020; Tang et al., 2020).

TPL detection is specifically designed to identify the diversity and version information of libraries integrated into target apps (Backes et al., 2016). Accurate TPL detection serves as a foundational prerequisite for numerous security-related studies, spanning multiple research domains including repackaged app inspection (Zhou et al., 2013; Wang et al., 2015; Zhan et al., 2019), library isolation (Liu et al., 2015), malware detection (Li et al., 2016), patch update (Duan et al., 2019; Derr et al., 2017; Huang et al., 2019), license violation detection (Duan et al., 2017), patch presence test (Zhang and Qian, 2018; Dai et al., 2020; Jiang et al., 2020; Xie et al., 2023; Xu et al., 2020; Pan et al., 2024; Zhan et al., 2024a,b) and software composition analysis (Jiang et al., 2023; Yuan et al., 2019; Jiang et al., 2024). Furthermore, accurate identification of TPL versions is increasingly crucial, as it facilitates the detection of potential vulnerabilities and serves as a foundational requirement for vulnerability-related research, including vulnerability identification (Bhoraskar et al., 2014) and patch presence test (Zhang and Qian, 2018). Vulnerability identification aids in detecting potential 1-day vulnerabilities within libraries, whereas patch presence test determines whether these vulnerabilities persist or have been addressed through security patches. Both tasks are essential for identifying security vulnerabilities in libraries. The continuous advancements in these areas underscore the indispensable role of TPL detection in securing the software supply chain.

2.2. Existing TPL detection methodologies

TPL detection approaches can be categorized into two types: clustering-based and similarity-based methods. Early efforts, such as LibRadar (Ma et al., 2016), LibD (Li et al., 2017b) and LibExtractor (Zhang et al., 2020), employed clustering-based techniques for TPL detection. This method first selects predefined code characteristics as features and then clusters them across thousands of apps to determine the optimal parameters for the clustering algorithm. Ultimately, TPLs can be detected through the trained clustering-based tools. However, several factors have led to the decline of clustering-based approaches.

First, these algorithms can only cluster commonly used TPLs present in their training data, which limits their ability to detect less prevalent libraries. Second, the similarity between different TPL versions varies significantly. For example, *Gson-2.8.5* and *Gson-2.8.6* exhibit high code similarity, whereas their similarity to *Gson-1.0* is below 40%. This variability complicates the determination of optimal clustering parameters, making it challenging to accurately categorize TPLs across different versions (Zhan et al., 2021b). Nevertheless, tasks like vulnerability identification rely on accurate library version detection as a prerequisite. Additionally, before using clustering-based tools, developers must collect millions of apps to fine-tune algorithm parameters. This is due to three main reasons: (1) similar to training neural network models, clustering requires a substantial amount of training data to extract meaningful code features, which are then used to group similar segments and identify potential TPLs. (2) the quality of clustering is highly sensitive to parameters such as similarity thresholds and cluster sizes. Developers must iteratively tune these parameters to effectively detect diverse TPLs with varying structural characteristics. (3) to assess the correctness of clustering results, developers are required to manually label and validate the generated clusters. Confirming the presence of actual TPLs through this labeling process is essential for guiding effective parameter tuning. Collectively, these limitations render clustering-based tools both time-consuming and labor-intensive, contributing to their gradual displacement by more efficient similarity-based approaches.

In similarity-based methods, features are first extracted from both the TPL and the target app. These extracted features are then compared pairwise to determine whether the TPL is present in the app. Compared to clustering-based methods, similarity-based approaches offer more direct, specific, and detailed feature representations, enabling them to accurately distinguish between and identify specific library versions with enhanced precision. Additionally, the feature characteristics of similarity-based approaches allow for the formulation of additional rules to mitigate the impact of obfuscation and optimization. To assess the prevalence of clustering-based and similarity-based methods, we collected a total of 16 TPL detection studies through keyword searches and other approaches from major publishers and digital libraries (Please see Section 3 for details). Among them, only three employed clustering-based methods, while the remaining 13 studies adopted similarity-based approaches. In addition, among the studies published in top-tier venues (clarivate, 2025; CSRankings, 2025), only one adopted a clustering-based method, whereas eight employed similarity-based approaches. This trend suggests that similarity-based methods may be more effective and better suited for TPL detection tasks.

2.3. TPL detection process

We provide a summary of the detection pipeline and failure modes of existing TPL detection approaches, as illustrated in Fig. 1.

(1) Preprocessing. This initial phase encompasses essential preparatory tasks, including the conversion of library *JAR* files into the *DEX* format and the utilization of decompilation tools, such as Androguard (2025), to transform both TPL and app code into suitable Intermediate Representations (IRs). These transformations lay the groundwork for

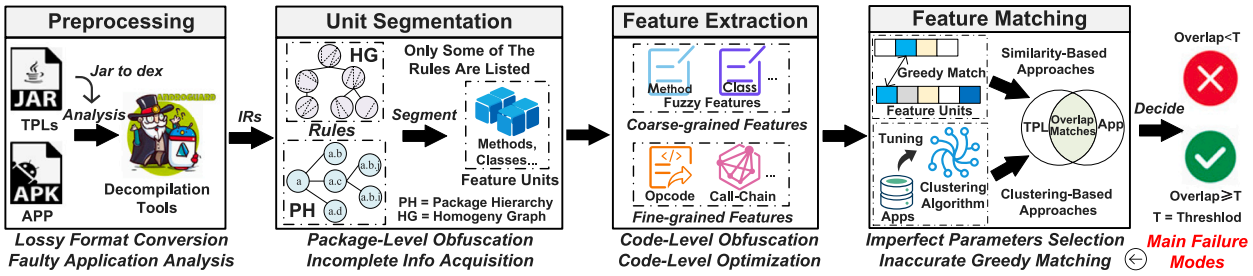


Fig. 1. Main processes of TPL detection approaches (we only introduce some of the techniques involved in each process).

efficient and accurate downstream analysis. However, this stage is prone to two primary issues that can lead to detection failure: *lossy format conversion* and *faulty application analysis*. The former typically stems from the inability to fully and faithfully convert instructions between *JAR* and *DEX* formats, which may result in altered or incomplete code features. The latter arises from limitations in decompilation tools, which often lack support for the latest Android versions, leading to partial or incorrect analysis of app code.

(2) Unit Segmentation. This step identifies the fundamental feature unit for comparison. The presence of a TPL is determined by assessing whether the ratio of matched feature units to total TPL feature units exceeds a predefined threshold. Some studies segment distinct TPL modules within an app (i.e., TPL module decoupling) by analyzing interrelationships in the app's code using techniques such as homogeny graph and class dependency. TPLs are then detected by comparing the similarity between the extracted module features and the original TPL features. In contrast, many recent studies bypass TPL module decoupling, avoiding the separation of library code from apps due to its incompleteness and susceptibility to errors. Instead, they utilize the app's package structure to distinguish package hierarchies or directly treat each class or method as a unit for subsequent feature matching. The main causes of inaccurate detection in this stage are *package-level obfuscation* and *incomplete information acquisition*. Package-level obfuscation techniques (also known as package obfuscation), such as package flattening and repackaging, can directly modify the package structure, thereby undermining approaches that rely on the package hierarchy for unit segmentation. Other segmentation strategies, such as those based on class dependencies or those that treat individual classes or methods as independent units, may also lead to the problem of *incomplete information acquisition*. Specifically, the former often fails to reconstruct a complete TPL unit set, thereby preventing the extraction of the entire TPL feature representation. The latter, in contrast, loses important contextual information – such as the position of a class within the original package hierarchy – the absence of which may result in inaccurate matching.

(3) Feature Extraction. During this phase, predefined features are systematically extracted from the target app based on the intermediate representations and segmented units. These features are often designed to be resilient to obfuscation and adaptable to advanced code optimizations. To enhance the comparison process, recent approaches frequently adopt a two-stage detection strategy that combines coarse-grained and fine-grained features. The coarse-grained features help eliminate dissimilar candidates, while the fine-grained features enable precise differentiation between TPL versions. At this stage, the primary factors contributing to detection failure are *code-level obfuscation and optimization*. These techniques can alter both the structural and syntactic representation of the code, resulting in inconsistencies between the TPL features present in the obfuscated app and those of the original TPL. Such discrepancies can prevent detection tools from correctly identifying the corresponding TPL.

(4) Feature Matching. Different detection approaches employ distinct strategies at this stage. Clustering-based approaches perform pre-clustering on large-scale apps to extract features of commonly used TPLs and subsequently apply a fine-tuned detection tool to identify TPLs within the target app. In contrast, similarity-based approaches begin by constructing feature profiles for a large number of TPLs. These profiles are stored in a database and later used for comparison with the features extracted from the app. If the similarity between an app and a TPL exceeds a predefined threshold, the TPL is reported as present, with the version exhibiting the highest similarity identified as the one used in the app. For clustering-based methods, the main challenge at this stage lies in the difficulty of selecting optimal parameters for the clustering algorithm, a problem known as *imperfect parameter selection*, as the parameters must effectively distinguish between different TPL variety while also identifying different versions of the same TPL. For similarity-based approaches, the key issue lies in the limitations of greedy matching strategies widely used by existing tools (*inaccurate greedy matching*). These strategies often result in feature unit mismatches, as similarity alone is often insufficient for accurately identifying true matches. Although some approaches attempt to introduce additional matching constraints beyond similarity to improve detection accuracy, this often significantly *increase detection time*, thereby reducing the efficiency and scalability of the tool.

2.4. Obfuscation and optimization

TPL detection relies on extracting code features from libraries to identify similar patterns within the target app, thereby assessing the likelihood of TPL presence based on feature similarity. However, external factors such as obfuscation and optimization can alter an app's code features, posing significant challenges to accurate detection. Therefore, this section summarizes common obfuscation and optimization techniques addressed in existing research, examines their effects, and delineates their impact scope.

Code obfuscation is a software development practice that deliberately reduces code readability while maintaining its original semantic functionality. Allatori (Allatori, 2025) and DashO (PreEmptive, 2025) have been consistently adopted as mainstream Android obfuscators in top-tier TPL detection studies, such as LibScout (Backes et al., 2016), LibID (Zhang et al., 2019), ATVHunter (Zhan et al., 2021a), LibLoom (Huang et al., 2022), and LibScan (Wu et al., 2023). Prior work has systematically characterized the obfuscation techniques implemented by these two obfuscators, thereby underscoring the significant challenges they pose to reliable TPL detection. Table 1 summarizes the common techniques identified in these studies that a robust TPL detection tool should be resilient against. Since these techniques have been well-documented in existing studies, we provide their detailed descriptions in the Supplementary Material.

Code optimization enhances program efficiency by refining code structure and expressions to minimize resource consumption. In Android development, code optimization primarily occurs in two stages: the static phase, during the compilation of bytecode (`.class`) into the app package (`.apk`), and the dynamic phase, which takes place at runtime. Since TPL detection operates on APK files, it is primarily influenced by static optimization. There are multiple optimizers capable of performing static optimization; however, existing research primarily

Table 1
Techniques employed by common Android obfuscators and optimizers (Guardsquare, 2025; Allatori, 2025; PreEmptive, 2025; Google, 2025a).

Obfuscation & Optimization	Allatori	DashO	ProGuard	R8 Compiler
Identifier Renaming	✓	✓	✓	✓
Repackaging	✓	✓	✓	✓
Package Flattening	–	✓	✓	✓
Dead Code Removal	✓	✓	✓	✓
String Encryption	✓	✓	–	–
Control-Flow Randomization	✓	✓	–	–
Method Inlining	–	–	✓	✓
Call-Site Optimization	–	–	✓	✓

focuses on ProGuard and R8, as they are the two generations of optimizers integrated into Android Studio (Google, 2025b). ProGuard provides both obfuscation and optimization for Java projects. R8, in addition to these capabilities, converts Java bytecode into DEX code and incorporates more optimization techniques. Since Android Gradle Plugin (AGP) version 3.4.0 (Google, 2025c), R8 has replaced ProGuard as the default optimizer. By default, ProGuard and R8 refrain from applying obfuscation and optimization unless explicitly configured by the user. This restriction arises because compile-time optimizations may cause crashes or errors if the configuration file is not properly written.

The key distinction between ProGuard and R8 lies in their optimization strategies. Although both tools can perform obfuscation on apps, ProGuard requires more explicit user configuration to specify the types and scope of optimizations, whereas R8 applies more advanced optimization techniques by default. As a result, ProGuard is also regarded as an obfuscator, while the code optimization discussed in current studies (Wu et al., 2023; Xie et al., 2024) specifically refers to R8's optimizations. Among these advanced optimizations, method inlining and call-site optimization have been incorporated into existing research due to their significant impact on TPL detection accuracy (Xie et al., 2024).

(1) Method Inlining. This optimization replaces a method call with the actual body of the method. As a result, inlined methods may integrate features from multiple methods. This transformation alters various TPL detection features, including opcodes and string constants, thereby affecting all detection approaches that rely on these features.

(2) Call-Site Optimization. This technique enhances method performance by modifying parameters and return values, thereby reducing execution overhead. For example, if a method's parameter is consistently assigned the same constant value across all its call sites within the entire app, R8 will replace the parameter with the constant itself. Moreover, if the return value of a method is not used at all, the return type of the method can be changed to void. In addition, R8 can eliminate parameters that are not utilized within a method. Since call-site optimization directly alters method signatures, it significantly impacts detection approaches that rely on method signatures as key features.

2.5. Related technologies of TPL detection

During our investigation, we found that certain technologies are not only highly relevant to the TPL detection research, but in some cases also inherently incorporate it. Therefore, in this subsection, we introduce these related techniques in the context of Android TPL detection to deepen readers' understanding of the research landscape.

Software Composition Analysis (SCA). SCA is a technique used to analyze known vulnerabilities in the open-source dependencies of a software (Zhao et al., 2023a; Imtiaz et al., 2021). SCA methods are typically classified into static and dynamic approaches (Github, 2025a). Common SCA tools (OWASP, 2025; Synk, 2025; Dependabot, 2025; Sonatype, 2025) primarily operate in a static manner, scanning source code, build files (e.g., *build.gradle*, and dependency artifacts (e.g., *jar*, *aar* files) to identify open-source libraries and their corresponding versions. By referencing vulnerability databases such as the National Vulnerability Database (NVD), these tools assist developers in identifying security vulnerabilities and license compliance issues, thereby mitigating the risks associated with open-source usage. Dynamic SCA tools (Plate et al., 2015; Ponta et al., 2018, 2020) are often employed to complement static SCA by analyzing the target program. These tools first compare semantic features, such as Abstract Syntax Tree (AST), between the program and known vulnerable dependencies stored in databases, in order to identify potential security risks within software components. Then, these dynamic approaches determine whether the vulnerable dependencies are actually invoked by the target program, thereby assessing whether the security threat is truly present. TPL detection is a critical component of SCA, as accurately identifying third-party dependencies is essential for assessing the associated security risks. Modern Android TPL detection is typically conducted without access to the target app's source code and, as such, more closely aligns with the step of dynamic SCA that identifies TPLs through code features. Consequently, the semantic features such as AST utilized in dynamic SCA can also be effectively leveraged for TPL detection.

App Clone Detection. App clone detection techniques have been extensively studied to identify malicious and pirated apps. These methods compute similarities between apps using a variety of features, including opcodes (Hanna et al., 2013; Gonzalez et al., 2015), Program Dependency Graph (PDG) (Crussell et al., 2012, 2014), API calls (Wang et al., 2015), centroids (Chen et al., 2014), and other features that reflect code semantics (Akram et al., 2018). This task is closely related to TPL detection, as both typically rely on different forms of code similarity analysis. Consequently, there is significant methodological overlap between code clone detection and TPL detection approaches. For example, because TPLs can affect the accuracy of app clone detection, Wukong (Wang et al., 2015) treats API calls as features and uses clustering-based methods to filter out TPLs. This approach is highly similar to that employed by the TPL detection tool LibRadar (Ma et al., 2016). In addition, some app clone detection methods (Hanna et al., 2013; Gonzalez et al., 2015) employ opcodes as features, transforming them into hash indices to facilitate efficient comparison—an approach also widely adopted in TPL detection, including by tools such as LibD (Li et al., 2017b) and Han's tool (Han et al., 2018). Recent app clone detection studies have further advanced feature extraction techniques. For instance, some approaches construct centroids (Chen et al., 2014) to better represent Control-Flow Graph (CFG), enabling more efficient feature matching while preserving key structural characteristics of the app. Similar feature representations have also been adopted in TPL detection tools such as OSSPolice (Duan et al., 2017).

Library Reuse and 1-day Vulnerabilities Identification. Some works, such as those focused on library (or open-source software) reuse and 1-day vulnerability identification, explicitly adopt TPL detection as a core objective, with their methodologies centered around library identification. For instance, the 1-day security risk identification tool OSSPolice (Duan et al., 2017) assesses potential license violations and security threats by identifying both Java and native library versions within Android apps. The features it employs, fuzzy method signatures and string constants, are widely used in TPL detection tools such as LibScout (Backes et al., 2016), LibID (Zhang et al., 2019), and LibHunter (Xie et al., 2024). In addition, although some state-of-the-art tools for open-source software reuse detection and 1-day vulnerability identification operate primarily in the C/C++ domain Kim et al. (2017), Xiao et al. (2020), Woo et al. (2021a,b, 2022, 2023), Xu et al. (2025), their methodologies are nonetheless closely related to Android TPL detection in various aspects. For example, Centris (Woo et al., 2021b) and V1Scan (Woo et al., 2023) employs a specialized hashing technique (i.e., locality sensitive hashing) to represent library features, enabling robust detection of reused open-source software even after significant code modifications. This approach is conceptually similar to that of ATVHunter, which utilizes rolling hashing to reduce the

impact of code obfuscation on library feature recognition. Furthermore, VULTURE (Xu et al., 2025) identifies candidate TPLs in source code by hashing function-level representations, and subsequently assesses potential security risks by analyzing whether the vulnerable code is actually invoked by the target program and whether the corresponding security patches have been applied. Similarly, function-level features, such as fuzzy method signatures, are also widely used in TPL detection approaches such as LibScan (Wu et al., 2023) and LibHunter (Xie et al., 2024).

Other Related Technologies. Beyond the aforementioned approaches, several related research domains, including repackaged app inspection (Zhan et al., 2019; Zhou et al., 2012) and library isolation (Liu et al., 2015), treat TPLs as noise and therefore require identifying and separating them from the core application modules. For instance, LibSift (Soh et al., 2016) employs a package dependency-based algorithm for TPL module detection. Similarly, code dependency analysis (e.g., class dependency) for TPL module segmentation is implemented in TPL detection tools like LibExtractor (Zhang et al., 2020).

2.6. Related work

To the best of our knowledge, there are currently three survey papers related to TPL detection, all published by the same research team. For clarity, we refer to these publications as ASE20 (Zhan et al., 2020), TSE21-a (Zhan et al., 2021b), and TSE21-b (Zhan et al., 2021c). ASE20 (Zhan et al., 2020) is the earliest comprehensive survey of TPL detection techniques, analyzing the strengths and weaknesses of research published between 2016 and 2019. TSE21-a presents the first systematic literature review of Android-related TPL research, with TPL detection introduced as one of the key topics. TSE21-b is an extended version of ASE20, incorporating additional tools including OSSPoLICE (Duan et al., 2017), LibExtractor (Zhang et al., 2020), and LibRoad (Xu and Yuan, 2020), and offering more detailed and diverse analyses. Since TSE21-b builds upon and fully covers the content of ASE20 (Zhan et al., 2020), we do not further analyze ASE20 in this paper. Therefore, the following discussion focuses on TSE21-a and TSE21-b.

The TPL detection section of TSE21-a begins by reviewing tools developed between 2016 and 2020. It then categorizes existing approaches along three dimensions: the type of TPL analyzed (e.g., native vs. Java libraries), the type of TPL detection approach (e.g., clustering-based vs. similarity-based methods), and the detection granularity (e.g., TPL variety level vs. version level). The paper further examines common detection techniques, typical obfuscation strategies, and the impact of obfuscation on detection effectiveness. Finally, it discusses the respective limitations of clustering- and similarity-based approaches and summarizes key findings from the reviewed works. TSE21-b (Zhan et al., 2021c) argues that prior studies lacked in-depth analysis of existing TPL detection tools. In response, it provides a comprehensive review of all TPL detection techniques proposed between 2016 and 2020. The paper begins by summarizing the challenges faced by existing TPL detection approaches (e.g., obfuscation) and their general workflow, followed by a discussion of the implementation details and limitations of current techniques. A major distinction from TSE21-a (Zhan et al., 2021b) is the use of the ORLIS (Wang et al., 2018) dataset to conduct an empirical study evaluating five representative tools – LibRadar (Ma et al., 2016), LibScout (Backes et al., 2016), LibPecker (Zhang et al., 2018), ORLIS, and LibID (Zhang et al., 2019) – in terms of their effectiveness, efficiency and robustness against obfuscation. Lastly, the paper discusses potential directions for future research.

While prior surveys have laid a foundation for understanding TPL detection techniques, their analyses are limited to work published before 2021, making them unable to effectively capture recent advancements that have rapidly emerged and significantly improved detection performance. Moreover, existing surveys tend to provide only high-level overviews of specific detection techniques, lacking in-depth analysis or empirical evaluation of their effectiveness in critical aspects such as anti-obfuscation and version identification. Finally, the absence of large-scale benchmark datasets further restricts the comprehensiveness and reliability of their evaluations. In contrast, our work significantly extends and deepens the understanding of TPL detection research along the following key dimensions:

(1) Expanded Analysis Scope Existing surveys primarily focus on TPL detection techniques published before 2021. However, the years 2021–2024 have witnessed the emergence of several high-impact works such as ATVHunter (Zhan et al., 2021a), LibLoom (Huang et al., 2022), LibScan (Wu et al., 2023), and LibHunter (Xie et al., 2024), accounting for over 40% of all top-tier publications in this domain. These recent studies not only introduced novel detection technologies (e.g., set-inclusion relationship) but also demonstrated substantial advancements in tackling prevalent challenges such as resilience to common obfuscation techniques—a limitation that hindered earlier tools like LibScout and LibID, which are vulnerable to package-level obfuscation. The strategies adopted by newer tools to overcome these challenges deserve dedicated investigation. Furthermore, recent methods have incorporated finer-grained features, such as call-chains combined with opcodes, to improve the accuracy of version identification, while analysis of these features is absent in prior surveys. Notably, a recent study named LibHunter has initiated discussions on the impact of Android optimization on TPL detection, highlighting an emerging research direction with significant practical implications. However, this research direction remains absent in existing surveys. To address the aforementioned limitations, our survey expands the temporal scope to include publications up to 2024, enabling a more comprehensive analysis of emerging techniques and newly identified challenges. This provides readers with a more complete and up-to-date perspective on the evolution of TPL detection.

(2) Refined Analytical Granularity Recent surveys primarily focus on evaluating the overall performance of TPL detection tools, but fall short of conducting fine-grained analyses of the specific detection techniques employed within these tools. Since most tools integrate multiple detection features, such as string constants, opcodes, and call chains, evaluating them as a whole makes it difficult to isolate the individual strengths and limitations of each feature. To address this limitation, our study goes beyond tool-level comparisons by decomposing existing tools into their underlying techniques and evaluating each technique independently. In Section 5.3, we conduct technique-level evaluations under both code obfuscation and optimization scenarios to assess how each commonly used TPL detection technique performs in these conditions. Furthermore, we examine the contribution of each feature to version identification, analyzing its discriminative power in distinguishing specific TPL versions. This fine-grained analysis provides a deeper understanding of the effectiveness and robustness of individual techniques, helping to better identify those that are more resilient to obfuscation or optimization, or that exhibit stronger capabilities in version differentiation. It also supports the development of more adaptable and effective TPL detection approaches.

(3) Broader Dataset Coverage Compared to earlier surveys, our work introduces a larger and more diverse benchmark composed of five distinct datasets, including three ground-truth datasets and two real-world datasets. These datasets contain not only original apps but also their obfuscated and optimized variants, along with real-world samples collected from publicly available app markets (e.g., Google play). In contrast, prior studies have typically relied solely on the ORLIS dataset as the ground truth benchmark for evaluation. This broader dataset allows us to be the first to simultaneously assess the prevalence of obfuscation and optimization in closed-source apps, thereby enabling an objective analysis of whether addressing obfuscation and optimization is essential for TPL detection research. By evaluating the performance of each tool under both obfuscation and optimization transformations, our study provides critical analysis into their resilience against such

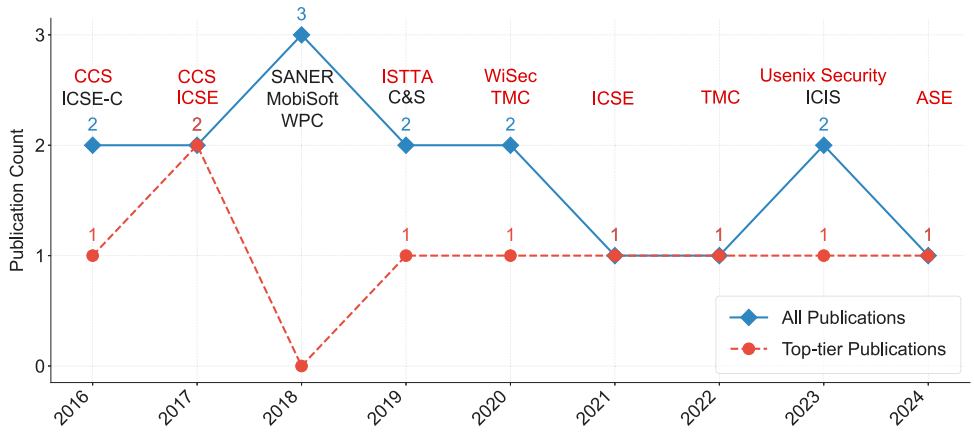


Fig. 2. Trend analysis of TPL detection publications from 2016 to 2024 (ICSE-C = ICSE-Companion, WPC = Wireless Personal Communications, C&S = Computer & Security, ICIS = International Conference on Information Security).

code modifications. The inclusion of two real-world datasets further enables a precise evaluation of tool performance in practical scenarios. In addition, by comparing performance across our diversity dataset, the tool's generalizability can be effectively demonstrated. Collectively, these advantages contribute to a more comprehensive and accurate evaluation than that offered by prior work.

3. Overview

We used a combination of keywords, such as **Android**, **third-party libraries detection**, **third-party libraries reuse**, **1-day vulnerability identification**, **license violation identification** and **software composition analysis**, to search for candidate papers in well-known digital repositories such as DBLP (dblp, 2025), Google Scholar (Google, 2025d), and other digital libraries like ACM. Since our study focuses on detecting TPLs in the Android domain using static analysis techniques, we exclude from consideration those papers that target the C/C++ ecosystem (Tang et al., 2020; Li et al., 2023) or rely on dynamic analysis methods (Cui et al., 2022). Through this process, we first collected nine relevant papers published in top-tier conferences and journals (clarivate, 2025; CSRankings, 2025), which represent some of the most influential works in this field. These works serve as the primary subjects of our evaluation analysis and include: LibScout (Backes et al., 2016), OSSPolice (Duan et al., 2017), LibD (Li et al., 2017b), LibID (Zhang et al., 2019), ATVHunter (Zhan et al., 2021a), LibRoad (Xu and Yuan, 2020), LibLoom (Huang et al., 2022), LibScan (Wu et al., 2023), and LibHunter (Xie et al., 2024). In addition, we believe that beyond the aforementioned publications, several other techniques also provide valuable insights and perspectives worth exploring. Therefore, we made a concerted effort to collect as many relevant studies as possible and ultimately identified seven additional approaches that offer valuable insights and complementary perspectives. These include LibRadar (Ma et al., 2016), LibPecker (Zhang et al., 2018), ORLIS (Wang et al., 2018), Han's tool (Han et al., 2018), PanGuard (Tang et al., 2019b), LibExtractor (Zhang et al., 2020), and Libra (Tomassi et al., 2023). Accordingly, we selected 16 papers as the target of our study (See Fig. 2).

Among these tools, LibScout, OSSPolice, LibID, LibLoom, LibScan, and LibHunter explicitly claim the capability to detect specific library versions. In contrast, clustering-based approaches such as LibRadar, LibD, and LibExtractor lack this ability, as clustering algorithms are inherently limited in identifying optimal parameters to distinguish between different TPL versions. Additionally, most of similarity-based methods such as LibScout, LibPecker, LibID, and LibRoad are capable of resisting common renaming obfuscation techniques. Meanwhile, ATVHunter, LibLoom, LibScan, and LibHunter assert robustness against all major forms of obfuscation. Notably, LibHunter further claims to be resilient to method inlining and call-site optimizations introduced by R8. The subsequent discussion will divide these works into three time periods at three-year intervals from 2016 to 2024, and provides a categorical overview of the foundational methodologies underlying these studies.

Papers in 2016–2018. This time period represents the early attempts in the field of TPL detection. LibRadar constructs feature vectors using Android APIs, the total number of APIs within each package, and their respective types. After pre-clustering millions of apps to optimize the clustering parameters, LibRadar can effectively detect commonly used TPLs. However, these features are too coarse-grained to enable detection of specific library versions. In contrast, similarity-based approaches have become the preferred choice in recent research due to their greater flexibility in fine-grained detection, as they are not constrained by the inherent limitations of clustering algorithms. This flexibility allows for fine-grained library version identification. LibScout (Backes et al., 2016) was the first approach to explicitly claim the capability of identifying specific library versions. It leverages fuzzy method signatures as features, where obfuscation-prone user-defined identifiers are replaced with placeholders, thereby enhancing resilience to common obfuscation techniques. Nevertheless, LibScout relies on a Merkle Tree (Merkle, 1987) constructed based on the package hierarchy, which renders it vulnerable to package obfuscation such as package flattening and repackaging. Another clustering-based approach, LibD (Li et al., 2017b), identifies TPLs by using hash values of opcodes extracted from Control-Flow Graph (CFG) as features. Compared to LibRadar, LibD's features are more distinctive, allowing for improved differentiation between various TPL variety. OSSPolice (Duan et al., 2017), the first approach capable of detecting both Java and native libraries, begins by identifying the presence of libraries using string constants and fuzzy method signatures. It also performs fine-grained analysis based on function centroids (Chen et al., 2014) to determine the specific TPL versions. Similar to LibD, Han's tool (Han et al., 2018) also utilize opcodes from basic blocks within the CFG as code features. However, unlike LibD, their approach identifies used libraries by comparing the pairwise similarity between the features extracted from the app and TPLs. ORLIS (Wang et al., 2018) first distinguishes individual TPL modules by analyzing class dependencies. It then indexes fuzzy method signatures using sdhash (sdhash@roussev.net, 2025), and generates class-level features using the ssdeep algorithm for subsequent pairwise similarity comparison. Another notable similarity-based tool, LibPecker (Zhang et al., 2018), utilizes class inheritance dependency, field dependency, and method invocation dependency as features. It generates class signatures based on class dependency features, incorporating all associated access flags, field signatures, and method signatures. In addition, it incorporates the package hierarchy to guide the feature matching process and assigns greater weights to classes with more distinctive features,

thereby enabling more accurate computation of similarity between the library and the target app.

Papers in 2019–2021. The tools developed during this stage surpass the previous approaches in terms of the complexity and diversity of both features and algorithms. PanGuard (Tang et al., 2019b) argues that existing detection tools suffer from low efficiency and are inadequate for handling code obfuscation and optimization introduced by ProGuard. It extracts the types of API calls from Android SDK as features, but unlike LibRadar, it does not consider the frequency of API occurrences, based on the rationale that the number of API calls may be altered by ProGuard transformations. Furthermore, PanGuard treats string constants as part of its feature set and adopts edge computing techniques to reduce the time required for greedy matching. LibID (Zhang et al., 2019) extracts fuzzy class features to capture fundamental code characteristics, incorporating fuzzy representations of the superclass, interfaces, fields, and method signatures. It also extracts opcodes from the CFG as fine-grained features. Additionally, to maximize the number of matches between the app and TPL while preserving package hierarchy constraints, LibID employs a novel Binary Integer Programming (BIP) algorithm. However, these constraints rely on the assumption that the package structure remains unchanged after obfuscation, which may not hold in scenarios involving package flattening or repackaging (Zhan et al., 2021b). As for LibRoad (Xu and Yuan, 2020), it leverages the package hierarchy tree as the primary structure for constructing library candidates. It extracts all method and field signatures from each class, concatenates them into a string, and then generates a final signature using an MD5 hash. Similar to most similarity-based approaches, it produces detection results by comparing the extracted features from the app with those from the TPLs. As a more recent clustering-based approach, LibExtractor (Zhang et al., 2020) employs various dependency relations – including superclass, interface, static field, instance field, direct method, and virtual method – as features. It constructs in-app TPL instances using an Inclusion-Dependency Graph (IDG), and then applies a clustering-based algorithm to perform large-scale analysis of TPL features for subsequent detection.

Papers in 2022–2024. With the increasing depth of research into obfuscation and optimization, the approaches developed during this stage assert their capability to handle all commonly encountered forms of obfuscation and optimization. To more effectively address package obfuscation, which can significantly impact approaches such as LibScout and LibID, ATVHunter (Zhan et al., 2021a) leverages class dependency relationships to distinguish between different TPL feature units, thereby reducing reliance on package structure. It assigns a unique serial number to each basic block within a method, transforming the basic blocks of the CFG into adjacency lists based on their execution order. Then, ATVHunter applies a sliding-window technique to divide the lists into smaller segments, mitigating localized feature changes introduced by obfuscation techniques such as control-flow randomization. However, similarity-based approaches must compare app features against a large set of TPL feature profiles to identify all potential libraries present in the app, making feature comparison efficiency a critical concern. To address this challenge, LibLoom (Huang et al., 2022) leverages a Bloom Filter (Wikipedia, 2025a) to represent obfuscation-resilient features, enhancing detection efficiency. Since LibLoom segments feature units based on package structure, it also incorporates an entropy-based heuristic algorithm to counter the effects of package obfuscation. LibScan (Wu et al., 2023) emphasizes precision in TPL detection. It introduces a set-inclusion relationship between obfuscated features and their original counterparts (i.e., the original feature is a subset of the obfuscated feature) and uses a two-stage detection process to improve effectiveness. In the first stage, fuzzy class features are used for coarse-grained filtering to identify potential matches. In the second stage, detection accuracy is further improved by comparing call-chain features between the matches. A feature unit is only considered a match if there is a set-inclusion relationship between the obfuscated app features and the original library features. The most recent study, LibHunter (Xie et al., 2024) highlights the impact of code optimization on TPL detection. It posits that opcodes and string constants can ensure better integrity and version distinction capabilities after common obfuscation and R8 optimization. Furthermore, LibHunter employs fine-grained static analysis to mitigate the effects of method inlining and call-site optimization, two optimization techniques that can significantly affect TPL detection approaches.

4. The underlying methodologies of TPL detection approaches

This section provides a systematic analysis of the fundamental methodologies employed by various tools at each stage of TPL detection. We begin with a comprehensive summary of all approaches, as presented in Table 2. We also provide an overview of the theoretical resilience of these TPL detection technologies against obfuscation and optimization techniques in Table 3. Following this, we conduct a detailed, stage-by-stage analysis of the techniques employed in each step of the detection process, evaluating the strengths and weaknesses of these technologies and explaining why and how they are vulnerable to prevalent obfuscation and optimization methods.

4.1. Methodologies in preprocessing

Various analytical tools for Android and Java facilitate the preprocessing stage, with nearly all TPL detection methods leveraging existing frameworks for this purpose. Among these, Androguard (2025) is the most widely adopted framework among the surveyed tools (7 out of 16). In contrast to general Java analysis tools such as Soot (Soot-oss, 2025) and WALA (2025), Androguard is specifically designed for analyzing *DEX* files, making it particularly well-suited for Android apps. It provides robust capabilities for constructing package hierarchies and performing static analysis, including CFG generation, method call-chain identification, and opcode sequence extraction. Additionally, it supports various types of dependency analysis, such as class dependency extraction. Another advantage of Androguard is its ability to directly extract Android opcodes, a capability not supported by typical Java decompilation frameworks such as Soot and WALA, which are limited to extracting Java opcodes. Furthermore, while tools such as Baksmali (JesusFreke, 2025) can convert *DEX* files into *smali* code, additional steps are required to extract the Android opcodes. Apktool (2025) also does not provide a direct interface for obtaining Android opcodes, further highlighting Androguard's unique advantage in this context. Despite its many advantages, Androguard has one notable limitation: since TPLs are commonly distributed in *JAR* format, they must first be converted to *DEX* format using tools such as dex2jar (pxb1988, 2025) before analysis. However, due to inherent differences between the instruction formats of *JAR* and *DEX*, *this conversion process may potentially alter code features*, thereby undermining detection accuracy. Soot (Soot-oss, 2025) is the second most commonly used framework among the examined tools (3 out of 16). As a mature Java analysis framework, Soot supports the extraction of fuzzy method signatures, string constants, and method call chains. By translating Java bytecode into a specialized intermediate representation known as Jimple, Soot is capable of directly analyzing both *APK* and *JAR* files without requiring format conversion. However, this design also imposes a limitation: Soot is unable to directly access Android opcodes, which hinders its capability to support Android opcode extraction, despite the widespread use of these opcodes in existing approaches such as LibScan and LibHunter. Another preprocessing framework, Apktool, integrates Baksmali to decompile *APK* files into *smali* code, thereby providing a foundation for subsequent extraction of TPL features such as class dependencies and package structures. Furthermore, Apktool can automatically extract and decode resource files within the *APK* and supports repackage operations, making it a comprehensive tool for app preprocessing. It is also important to note that these decompilation tools may *fail to correctly (or accurately) analyze certain apps that utilize higher versions of the Android API*, as the Android platform evolves more rapidly than the compatibility improvements of these tools. This limitation can also lead to detection failures.

Table 2
A comprehensive comparison of state-of-the-art TPL detection tools (P(D) indicates whether the prototype tool and dataset are publicly available; "-" indicates information not explicitly stated in the publication).

Tool	P(D)	Framework	Segmentation (Unit)	Feature Extraction (Representation)	Matching Strategy
LibRadar (16)	✓(×)	Apktool	Package Structure (Package)	Number and Kinds of API Calls (Hash) Kinds of API Calls (Hash)	Clustering Algorithm
LibScout (16)	✓(×)	Soot	Package Structure (Class)	Fuzzy Method Signature (Merkle Tree)	Greedy Matching
LibD (17)	✓(×)	Androguard	Homogeny Graph (Library)	Opcodes of CFG (Hash)	Clustering Algorithm
OSSPolice (17)	✓(×)	Soot	Package Structure (Class)	Fuzzy Method Signature (String) String Constant and Function Centroid (Set)	Greedy Matching
LibPecker (18)	✓(×)	Androguard	Package Structure (Class)	Class Dependency and Access Flag (Hash)	Weighted Greedy
ORLIS (18)	✓(✓)	Soot	Single Method (Method)	Fuzzy Method Signature (string) Call-Chain (string)	Greedy Matching Set-Inclusion Rules
Han's tool (18)	×(×)	Androguard	Package Structure (-)	Opcodes of CFG (Hash)	Greedy Matching
PanGuard (19)	×(×)	–	Package Structure (Package)	Kinds of API Calls (Hash) String Constant (Hash)	Greedy Matching
LibID (19)	✓(×)	Androguard	Package Structure (Class)	Fuzzy Class Feature (String) Opcodes of CFG (String)	BIP Algorithm
LibRoad (20)	×(×)	Apktool	Package Structure (Class)	Fuzzy Class Feature (Hash)	Weighted Greedy
LibExtractor (20)	×(×)	Baksmali	Class Dependency (Library)	Class Dependency (String)	Clustering Algorithm
ATVHunter (21)	×(✓)	Androguard	Class Dependency (Library)	Basic Block of CFG (Hash) Opcodes of CFG (Rolling Hash)	Greedy Matching
LibLoom (22)	✓(✓)	WALA	Package Structure (Class) Entropy-Based Rules	Fuzzy Class Feature (Bloom Filter) Package Feature (Bloom Filter)	Greedy Matching
Libra (23)	×(×)	–	Package Structure (Method)	Fuzzy Method Signature (String) Opcode of CFG (CTP Hash)	Greedy Matching
LibScan (23)	✓(✓)	Androguard	Single Class (Class)	Fuzzy Class Feature (Bloom Filter) Opcodes (Integer) and Call-Chain (String)	Greedy Matching Set-Inclusion Rules
LibHunter (24)	✓(✓)	Androguard	Single Method (Method)	Advanced Fuzzy Feature (Bloom Filter) String Constant and Opcodes (Integer)	Compensatory Greedy

4.2. Methodologies in unit segmentation

Segmenting feature units based on package structure is the most widely adopted method in TPL detection tools, used in 10 out of 16 approaches. The package hierarchy follows a tree-like structure, consisting of a root package and its sub-packages, characterized by multi-level parent–child relationships. Detection approaches can leverage this hierarchical organization to construct or compare features systematically. For example, package *a* serves as the root for package *a.b* and *a.b.c*, with package *a.b* acting as the parent of package *a.b.c*, and each package generally contains multiple classes of the similar functional type. Since TPLs typically appear in an app as a root package along with its sub-packages, most existing studies treat each root package as a potential TPL candidate. During feature comparison, packages at the same hierarchical level (e.g., *a.b* and *a.c*) within a root package are considered as potential match pairs and are subjected to similarity-based matching. The presence of a TPL in a target app is subsequently determined by evaluating whether the number of matching feature units (e.g., classes or methods) between the app's root package and the TPL meets a predefined threshold. Among the approaches that segment feature units using the package hierarchy, most tools, such as LibID, LibPecker, and LibLoom, select the class as the basic feature unit. This strategy is favored because classes typically contain rich semantic information, and class-level pairwise matching between the app and TPL incurs an acceptable computational overhead. Several tools also employ root packages to distinguish TPL candidates but adopt alternative granularities by treating either the package (e.g., LibRadar) or the method (e.g., Libra) as the basic feature unit. The former may suffer from overly coarse feature granularity, potentially overlooking subtle distinctions and losing the ability to identify specific library versions, while the latter may lead to low efficiency due to the significantly increased number of required feature comparisons. However, obfuscation techniques such as *package flattening and repackaging can significantly disrupt the original package hierarchy by relocating obfuscated classes to other packages*. This structural alteration undermines traditional package-based feature comparison, making it unreliable. For instance, a root package may originally contain multiple sub-packages such as *a.a*, *a.b*, *a.c*, each with their respective classes. After package obfuscation, only a subset of the original structure (e.g., *a.a* and its classes) may remain, resulting in an insufficient number of matching features to meet the similarity threshold required for detection.

To mitigate the impact of obfuscation while still leveraging package hierarchy as the basis for feature segmentation, LibLoom employs an entropy-based heuristic algorithm to detect potential package obfuscation and identify the corresponding destination packages (for more details about obfuscation and destination packages, please refer to the Supplementary Material). The key insight of this method is that destination packages often contain all sub-packages and their associated classes that have been relocated due to obfuscation. Since a distinct package typically encapsulates different functionalities, the type distribution of classes in this destination package exhibits higher information entropy compared to that in a regular, non-obfuscated package. By identifying the destination package with the highest entropy, LibLoom first performs similarity matching between non-obfuscated packages in the app and TPL packages based on the package hierarchy, thereby obtaining matched package pairs. For each these pair, it conducts class-level matching to ultimately determine the class matches. Next, LibLoom performs similarity matching between the classes within an app's destination packages and the remaining unmatched classes in the TPLs. This strategy enables LibLoom to effectively match TPL features that have been obfuscated through package-level restructuring. However, *this heuristic technique is not guaranteed to be always effective*, which may impact the general applicability of LibLoom.

Homogeny graph and class dependency are alternative methods designed to mitigate the impact of package obfuscation on feature unit segmentation. Compared to directly using the package hierarchy for segmentation, these two approaches construct TPL modules as feature units by leveraging interrelationships within the code. As obfuscation does not alter the semantics of the code, these approaches assert

that code interrelationships also remain consistent before and after obfuscation. LibD constructs a homogeny graph based on call relations, inheritance relations, and inclusion relations, where each node represents either a package or a class, and edges indicate inclusion or inheritance relationships between nodes. The homogeny graph reflects the internal dependency structure of a TPL, enabling the construction of a more complete TPL instance for subsequent feature extraction and comparison. On the other hand, some tools – such as LibExtractor and ATVHunter – leverage class dependency relationships to isolate independent TPL modules. These dependencies commonly include class inheritance, interface implementation, field references, and method invocations, although the exact definitions may vary slightly across tools. Class dependency relationships are not altered by changes in the package structure, making them robust against external transformations and well-suited for reliable TPL detection. Nevertheless, the homogeny graph still relies, to some extent, on package structures. As a result, *the homogeny graph is vulnerable to package obfuscation*. While class dependency is more resilient to common obfuscation technologies, *it may fail to generate accurate and complete TPL instances in some cases*. This is because certain classes, such as interfaces, which lack strong dependencies on other classes, may be omitted during unit segmentation, resulting in incomplete TPL features.

As observed, the aforementioned feature unit segmentation methods each suffer from certain limitations. Consequently, some studies – such as ORLIS, Libra, LibScan and LibHunter – adopt an alternative strategy by treating each class or method as an independent feature unit in TPL detection. The primary advantage of this approach lies in its robustness against package obfuscation (i.e., package flattening and repackaging), as it does not rely on hierarchical structures. However, this method also has notable drawbacks—*it loses certain contextual information*, such as the inability to leverage package hierarchies to map potential correspondences between packages in the target app and TPLs, which may reduce detection accuracy. For example, packages *a.a* and *a.b* share the same hierarchical structure and are therefore considered potential matches, whereas classes from *a.a* and *a.a.b* are clearly unlikely to correspond. Due to the absence of such contextual cues, tools like ORLIS, Libra, LibScan and LibHunter treat all methods (or classes) in both the app and the TPL as independent matching candidates for feature comparison. Nevertheless, method (or class) features may be similar for reasons such as inheritance, overriding, or reuse. As a result, these unit segmentation methods fail to leverage contextual information to constrain the matching scope of features, which may lead to additional false positives and false negatives.

4.3. Methodologies in feature extraction

The feature extraction process is the most critical component of TPL detection, as the selected features directly influence the effectiveness of the tool. We begin by presenting a detailed analysis of the feature extraction processes used by each tool and introduce the obfuscation and optimization techniques that may affect each feature at the point of its first mention. Subsequently, we summarize the additional limitations of each feature beyond their weaknesses in resisting obfuscation and optimization.

LibRadar aims to achieve fast and accurate TPL detection. To this end, it extracts the total number of Android API calls and the total number of distinct Android API types for each package, which are then hashed to generate a package-level fingerprint. LibRadar claims that these features are resistant to renaming obfuscation and effectively capture the low-level behavioral characteristics of TPLs. Although Android APIs are generally not affected by obfuscation and optimization, the coarse granularity of these features limits their ability to distinguish between different library versions, thereby significantly restricting the effectiveness of LibRadar in TPL detection.

LibScout utilizes fuzzy method signatures as its core feature. These fuzzy features enhance obfuscation resilience by replacing easily obfuscated identifier with placeholders. For example, LibScout abstracts all user-defined variables in method signatures as "X". Depend on these fuzzy signatures, LibScout incrementally constructs a Merkle Tree based on the package hierarchy, where the value of each parent node is computed from the aggregated features of its child nodes. Leveraging this hierarchical construction, LibScout follows a step-by-step aggregation process: it first constructs class-level features by aggregating method features within the same class, then derives package-level features by combining features from same-level classes, and finally builds app or TPL features by integrating package features. The limitation of the fuzzy method signature approach lies in its inability to resist call-site optimization, as this transformation can alter method parameters and return types. Furthermore, code shrinking and method inlining can also affect the effectiveness of fuzzy method signatures by reducing the overall number of methods.

Table 3
Detection techniques and their resilience to obfuscation and optimization (symbol "–" indicates that the technique is not strongly affected by the corresponding obfuscation or optimization, symbol "×" indicates that the technique is vulnerable to the technology, while symbol "?" indicates that this technique requires certain prerequisites to be met in order to avoid interference from corresponding techniques. RP = Repackaging, PF = Package Flattening, IR = Identifier Renaming, SE = String Encryption, CFGR = Control-Flow Graph Randomization, MI = Method Inlining, CSO = Call-Site Optimization.).

Technologies	RP	PF	IR	CS	SE	CFGR	MI	CSO
Package Structure	×	×	–	–	–	–	–	–
Homogeny Graph	×	×	–	–	–	–	–	–
Class Dependency	–	–	–	–	–	–	–	–
Entropy-Based Rules	?	?	–	–	–	–	–	–
Single Class/Method	–	–	–	–	–	–	–	–
Number and Type of API	–	–	–	–	–	–	–	–
Fuzzy Method Signature	–	–	–	×	–	–	×	×
Fuzzy Class Feature	–	–	–	×	–	–	×	×
Package Feature	?	?	–	×	–	–	×	×
String Constant	–	–	×	–	×	–	–	–
Function Centroid	–	–	×	–	–	×	×	–
Class Dependency	–	–	–	×	–	–	×	×
Basic Block of CFG	–	–	–	×	–	×	×	×
Opcodes of CFG	–	–	–	×	–	×	×	×
Call-Chain Feature	–	–	–	–	–	×	×	×
Advanced Fuzzy Feature	–	–	–	×	–	–	–	–

LibD constructs its features by performing a depth-first traversal of the CFG, processing each basic block in the homogeny graph and extracting the hash values of their opcodes. These hash values are then concatenated sequentially to form the feature representation. Nevertheless, LibD's feature – opcodes of the CFG – can be significantly affected by code shrinking and control-flow randomization. The former compromises the completeness of opcodes by removing unused code and merging related statements, while the latter introduces new conditional statements, which can increase both the number and variety of opcodes in the obfuscated apps. Moreover, optimizations such as method inlining and call-site optimization can also affect the opcode features.

OSSPolice initially extracts fuzzy method signatures and string constants as part of its fingerprinting process, and further augments its feature set by incorporating the function centroid as an additional feature. A function centroid is computed through a deterministic traversal of the intra-procedural CFG, capturing key control flow characteristics of a function. Each centroid is represented as a three-dimensional data point, encoding the basic block index, outgoing degree, and loop depth. This representation allows OSSPolice to more accurately reflect the unique behavioral semantics of a function, thereby increasing the specificity and distinctiveness of the extracted features. However, the string constant feature can be fundamentally altered through string encryption obfuscation, and the function centroid is vulnerable to obfuscation and optimizations that affect the number of basic blocks and the opcodes they contain, such as code shrinking, control-flow randomization, and method inlining.

LibPecker proposes class dependency features to enhance robustness against obfuscation, including class-inheritance dependency, field-in dependency, and method-prototype dependency. A class-inheritance dependency exists between classes x and y when y is the superclass of x. Additionally, if class x contains a field whose type is class y, class x is considered field-dependent on class y. Similarly, a method-prototype dependency is defined when a method in class x uses parameters or return types that match those of class y, making class x method-prototype dependent on class y. LibPecker further collects the access flags of classes, fields, and methods, and incorporates them into the class signature alongside the class dependency features. Although the representation of dependency relationships and certain access flags typically remains unchanged by common obfuscation techniques, code shrinking and method inlining can reduce the number of dependency relationships, thereby impacting their effectiveness. Additionally, optimizations such as call-site optimization can alter the expression of class dependency features by modifying method parameter types and return value types.

ORLIS extracts fuzzy method signatures as part of its class-level features. It assumes that the method call relationships between library methods and their counterparts in the app will be preserved, regardless of whether the app has been obfuscated. Since obfuscation may introduce junk call sites to a given method, ORLIS posits that if a library method m is present in an obfuscated app, its set of callees in the library will likely be a subset of those in the obfuscated version of method m'. Therefore, for each method m, ORLIS concatenates its fuzzy method signature with the signatures of all its callees into a single string (i.e., a call chain) to enhance the accuracy of its features. However, the method call-chain feature can be affected by method inlining and call-site optimization. The former can reduce the number of call sites within an app method, thereby affecting the completeness of the call chain, while the latter may alter method signatures, thus impacting the representation of the call chain.

Han's tool, similar to LibD, extracts the hash value of opcode sequences from a method's CFG as its core feature. The class-level hash is then computed by hashing the sequence of its constituent method-level hashes. Using the same strategy, package-level features are constructed by aggregating and hashing the class-level features, enabling hierarchical feature representation from method to package. Please refer to the entries on **LibD** for a more detailed analysis of the feature used in Han's tool—namely, the opcode of CFG.

PanGuard adopts an approach similar to that of LibRadar, initially extracting the types of API calls from the Android SDK. To enhance scalability and improve resilience against code transformations introduced by ProGuard, PanGuard considers only the types of APIs used, while deliberately disregarding their invocation frequency. Additionally, PanGuard also extracts string constants from each method as part of its feature set. The feature used by PanGuard is susceptible to obfuscation and optimization techniques such as code shrinking, method inlining, and string encryption.

LibID extracts fuzzy class features to capture fundamental code characteristics, incorporating fuzzy representations of the superclass, interfaces, fields, and method signatures. Additionally, it traverses all basic blocks of the CFG in each method to extract Android opcodes. The extracted features within the same class are then aggregated into a set, serving as the class-level representation. Fuzzy class features can be affected by code shrinking, which may remove unused fields, methods, and other elements. They are also susceptible to method inlining and call-site optimization, as both techniques can alter the set of method signatures contained within the class. The opcodes of the CFG, on the other hand, are also susceptible to certain obfuscation and optimization techniques, as previously mentioned in **LibD's** entries.

LibExtractor extracts six types of dependencies from each class file as features, including superclass, interface, static fields, instance fields, direct methods, and virtual methods. First, it computes the relative path from the current class to each of its dependency classes. For example, if a class within the package hierarchy *a.a.a* depends on another class located in *a.b.c*, the original relative path is *b.c*. To enhance resilience against obfuscation, the package names *b* and *c* are replaced with placeholders. The relative paths of all six dependency types for a class (e.g., x) are then extracted, sorted by their length, and concatenated into a single string, which is subsequently hashed to obtain the initial feature. In the second step, for each dependency class of a given class x, its corresponding initial feature (as extracted in the first step) is retrieved and concatenated into a new string. The concatenated strings for the six dependency types are then merged into a final string and hashed to produce the final feature representation.

ATVHunter utilizes both coarse-grained and fine-grained features. It first traverses the CFG, assigning a unique serial number n to each basic block and storing its successor relationships (parent n -> child1, child2) in a list. The hash value of this list serves as the coarse-grained feature. For fine-grained features, ATVHunter processes the opcodes of each basic block using a rolling hash, which applies a sliding window mechanism to segment strings and generate fingerprints, enabling more precise comparative analysis. However, both its coarse-grained features (i.e., basic blocks of the CFG) and fine-grained features (i.e., opcodes within the CFG) can be modified by code shrinking and CFG randomization, as these techniques may increase or decrease the number of basic blocks and opcodes. Furthermore, these features are also susceptible to method inlining and call-site optimization—two common compiler optimizations that can alter the structure of basic blocks and the sequence of opcodes within a method.

LibLoom employs a two-phase detection mechanism. Initially, it employs fuzzy class features, similar to LibID, as primary attributes. It then builds package-level features based on package hierarchy, destination package and class features to exclude invalid match targets. LibLoom argues that finer-grained features (e.g., opcodes) are more vulnerable to obfuscation and fail to meet the scalability requirements of TPL detection, making them impractical for real-world apps. Consequently, unlike LibID and ATVHunter, it does not perform static analysis to extract opcodes or call-chain features. To enhance efficiency, LibLoom maps features into bloom filter indices by hashing elements into a bit array of length m using multiple hash functions. This approach enables feature comparison with a time complexity of $O(m)$, significantly improving efficiency. However, LibLoom's package feature is built upon the fuzzy class feature and is therefore similarly affected by code shrinking, method inlining, and call-site optimization. In addition, if its entropy-based approach fails to accurately identify the destination package, the package feature will also be vulnerable to package obfuscation such as package flattening and repackaging.

Libra divides each method feature into two components: the header and the body. The header consists of the method's fuzzy signature, while the body is composed of its instruction opcodes. To facilitate more effective comparison, Libra transforms the body using the context-triggered piecewise hashing (CTP hash) algorithm (Kornblum, 2006), which enables similarity-based matching of opcode sequences in a more robust and efficient manner. For an analysis of the obfuscation and optimization resilience of Libra's feature (i.e., fuzzy method signatures and opcodes of CFG), please refer to the entries on **LibScout** and **LibD** in this subsection.

LibScan utilizes two distinct types of features. Initially, it enumerates fuzzy class features that can be expressed in boolean form, such as whether a class is abstract, to filter out invalid matching candidates (i.e., classes). Subsequently, it constructs fine-grained features by aggregating opcode sequences and call-chains (represented as fuzzy method signatures for the callees of each method), thereby achieving higher feature accuracy. For an analysis of the obfuscation and optimization resilience of LibScan's features, including opcodes of the CFG, fuzzy class feature and call-chain, please refer to the entries on **LibD**, **LibID** and **ORLIS**.

LibHunter follows a similar process to LibScan. It initially filters potential matching class pairs based on the fuzzy features of fields and

method signatures. Subsequently, it further enhances feature granularity through the analysis of opcode sequences and string constants, enabling precise identification of correct feature mappings between the app and TPL. Compared to other approaches, LibHunter performs additional fine-grained static analysis to generate advanced fuzzy features by identifying library methods that may be affected by method inlining and call-site optimization performed by the R8 optimizer. Specifically, it targets methods likely to be inlined, enhancing their features by integrating their opcodes and string constants with those of the methods that inlining them. Additionally, it replaces identifiers potentially altered by call-site optimization with a placeholder symbol "?", which matches any identifier, thereby enhancing robustness against such optimization. As a result, it is only affected by the loss of feature completeness caused by code shrinking.

Based on the above analysis, *it can be concluded that no single feature currently possesses both resilience against all obfuscation and optimization techniques and the capability for accurate version identification*. Moreover, these features exhibit several additional limitations. Among the various methods used in TPL detection, fuzzy features and opcodes of CFG are the two most prominent. The former is employed as a core attribute in 9 out of 16 approaches, while the latter is incorporated into the feature sets of 7 out of 16 studies. Fuzzy features enhance resilience against identifier renaming – which is widely employed by various obfuscators and optimizers – by generalizing identifiers that are susceptible to obfuscation and optimization transformations. However, this generalization comes at the expense of distinctiveness: placeholders can substitute for any identifier, thereby increasing the risk of mismatches during comparison and potentially diminishing detection effectiveness. Opcodes offer significant advantages in terms of comparison efficiency. Nevertheless, they inherently lack contextual and semantic information. For instance, two methods with completely different functionalities may share identical opcodes, limiting their discriminative power. This issue is particularly evident in approaches that transform opcodes into sets of integers, focusing only on opcode types while ignoring their frequency of occurrence. The call-chains employ by ORLIS and LibScan can capture the order and behavior of code execution within a method, acting as a simplified version of the CFG by recording only the fuzzy method signatures of callees. This feature provides concise yet informative representations, making it more resistant to obfuscation and optimization. Although optimizations such as method inlining and call-site optimization can modify call-chain features, our analysis (see Section 5.3.2) indicates that the impact is relatively limited. Its main drawback lies in the fact that its extraction process requires detailed static analysis, which is time-consuming and can significantly degrade the efficiency of detection tools. In addition, both OSSPolice and LibHunter utilize string constants within methods as features; unfortunately, these features are not unique, as different methods may contain identical string constants. For example, in the TPL *bcprov-jdk15on*, the same long string "this cipher needs algorithm parameters for initialization..". appears more than six times, while simpler strings are even more likely to repeat. This phenomenon, undoubtedly, may lead to false positives in detection. Similar to the call-chain feature, both the function centroid and advanced fuzzy features require sophisticated static analysis for extraction, which directly impacts the efficiency of detection tools. However, compared to call-chain features, the extraction processes for these two features are more complex, as existing decompilation frameworks such as Androguard and Soot do not natively support the generation of function centroids or advanced fuzzy features. Furthermore, the API feature and class dependency are constrained by the granularity of detection. The API feature typically exhibits noticeable differences only when there are significant changes in the types or number of APIs used; however, updates between different versions of a library may involve only minor modifications. Similarly, even when a library undergoes substantial functional updates, its class dependency structure may remain unchanged. These factors make it difficult for either feature to effectively distinguish between different versions of the same library. Finally, as mentioned earlier, the package feature employed by tools such as LibLoom relies on the prerequisite that TPL detection tools are resilient to package obfuscation, for example, by accurately identifying the destination package. Nevertheless, current destination package identification techniques primarily rely on heuristic algorithms, which cannot guarantee consistent effectiveness. This limitation potentially undermines the robustness of the package feature.

4.4. Methodologies in feature matching

Most approaches (12 out of 16) adopt a greedy matching algorithm for feature matching, where app feature units are paired with TPL feature units based on the highest similarity score. This method is both intuitive and relatively efficient. However, similarity alone is insufficient to determine whether two features truly match, especially considering that the similarity between corresponding features can be significantly affected by obfuscation and optimization. In such cases, *relying solely on similarity without incorporating additional matching constraints substantially increases the risk of mismatches*. Although both LibPecker and LibRoad adopt a greedy matching algorithm, they recognize that different classes contribute unequally to the determination of TPL presence. Specifically, both tools observe that when a class with a larger number of member methods is successfully matched, the likelihood of the corresponding TPL being present significantly increases. As a result, LibPecker assigns higher weights to classes with more methods during feature matching. Apart from this, LibRoad increases the weight of packages containing more classes once a match is found. This weighting strategy effectively mitigates the issue of false negatives caused by code shrinking. However, the core of their approach remains greedy-based matching, which often leads to suboptimal results due to potential mismatches. LibID addresses this issue by incorporating two additional rules: class dependency rules and package hierarchy rules. By leveraging a Binary Integer Programming (BIP) algorithm, LibID maximizes match pairs that adhere to these rules. However, its effectiveness relies on the assumption that obfuscators and optimizers do not modify the package structure of apps. Techniques such as package flattening and repackaging can disrupt this assumption. Additionally, solving the BIP problem is NP-hard, *making it computationally expensive and more suitable for small-scale scenarios rather than large-scale TPL detection*, which requires efficient matching algorithms. LibHunter, by contrast, introduces a two-stage matching strategy. In the first stage, it performs greedy matching between app method features and the original TPL method features, similar to other approaches such as LibLoom. The key difference lies in its second stage: for the remaining unmatched app methods, LibHunter further employs a compensation mechanism to match them with TPL methods using advanced fuzzy features. This allows the detection of feature pairs that may have been missed due to code optimizations. However, since advanced fuzzy features are generalized representations that inherently lack some degree of distinctiveness, the compensatory matching may introduce additional matched features, regardless of whether they truly correspond to correct matching pairs. *This can result in an increased number of false positives*. Moreover, the compensation process incurs additional detection time, thereby reducing the overall efficiency of the method. The clustering-based approaches such as LibRadar, LibD and LibExtroctor employ a pre-trained clustering algorithm to detect TPLs, eliminating the need to construct a TPL feature database for comparison. This approach provides an alternative to traditional similarity-based methods. Nevertheless, *selecting optimal parameters for the clustering algorithm remains challenging*, as it must accommodate the wide variety of TPLs and their different versions. Feature similarities can vary significantly—not only across different TPLs, which may exhibit either substantial similarity or complete dissimilarity, but also among different versions of the same TPL, where the degree of similarity can range from very high to relatively low. To account for

obfuscation techniques that introduce junk conditions and spurious method calls, ORLIS enforces a set-inclusion relationship on call-chain features, while LibScan additionally applies a set-inclusion relationship on opcode sequences as an extra constraint. Specifically, the features of a library method must form a subset of the features of the corresponding app method. By applying these set-inclusion rules, ORLIS and LibScan refine the matching results obtained through the greedy matching algorithm. Since optimization can alter method signatures by modifying function parameters and return types, it may disrupt the set-inclusion relationship, thereby potentially reducing detection accuracy.

5. Empirical study

Based on the summarized methodologies of existing TPL detection approaches, this section presents a comprehensive evaluation structured around the following key research questions:

- **Q1:** What is the prevalence of obfuscation and optimization in real-world close-source apps?
- **Q2:** How effective are existing approaches? This includes an in-depth analysis of four subtopics: (a) variety detection effectiveness, (b) obfuscation- and optimization-resilience, (c) version identification capability, and (d) false positive and false negative analysis.
- **Q3:** What is the efficiency of current approaches?
- **Q4:** How generalizable are existing approaches when applied to diverse scenarios?
- **Q5:** How do current approaches perform on real-world apps?

Our research questions cover all key aspects of TPL detection tools, including effectiveness, generalizability and efficiency, ensuring a comprehensive evaluation. The rationale for selecting these questions is as follows: (1) Although prior research has investigated the prevalence of code obfuscation, to the best of our knowledge, no study has systematically examined the use of R8 optimization techniques in the wild (e.g., closed-source apps distributed through Google Play). Therefore, Q1 aims to jointly consider both obfuscation and optimization, investigating their prevalence in real-world apps. This investigation is intended to assess whether resilience to such transformations is truly essential for effective TPL detection. (2) Building on this, Q2 evaluates the practical effectiveness of TPL detection tools using three ground-truth datasets, highlighting their capabilities in realistic scenarios. Given that library version identification and resilience to obfuscation and optimization are central themes in recent studies (Wu et al., 2023; Xie et al., 2024), it is necessary to examine how well the core detection techniques employed by these tools address these challenges, thereby offering deeper explanations for their observed performance. In addition, we investigate the causes of false positives and false negatives to provide actionable insights for improving future detection strategies. (3) To support supply chain security and respond quickly to emerging security threats, TPL detection tools must also be efficient. Thus, in Q3, we evaluate the runtime performance of these tools across multiple dimensions and at different analysis stages. (4) The generalizability of a tool is a crucial factor in evaluating its practical reliability. In Q4, we examine the performance variations of each tool across multiple datasets and analyze the underlying reasons for these fluctuations. Furthermore, we compare our empirical findings with the results reported in the original publications of the respective tools to identify discrepancies and explore their possible causes. (5) The real-world experiment serves as an irreplaceable means of evaluating the practical value of current approaches. Accordingly, in Q5, we assess and analyze the effectiveness and efficiency of existing tools when applied to real-world Android apps.

5.1. Experiment settings

Before commencing the evaluation, it is essential to first select the comparison tools, collect the datasets, and introduce our testbed. These components are described in detail as follows.

Selected Tools. We evaluate all publicly available tools published in top-tier venues listed in Table 2, with the exception of LibD, OSSPolice, LibRoad and ATVHunter. LibD and OSSPolice have been reported by their original authors to contain functional flaws (IIE-LibD, 2025; OSSSanitizer, 2025) and are difficult to reproduce, while LibRoad and ATVHunter are excluded due to its unavailability to the public. Consequently, our comparative analysis includes the following tools: LibScout (Backes et al., 2016), LibID (Zhang et al., 2019), LibLoom (Huang et al., 2022), LibScan (Wu et al., 2023), and LibHunter (Xie et al., 2024). All tools were executed using their original implementations without any modifications, in order to preserve the integrity of their design and avoid unintended deviations from the authors' intended behavior.

Datasets and Testbed. To address all the aforementioned questions, we collected five datasets, which include: (1) Dataset 1: 50,000 apps from Google Play (99% app is close-source). (2) Dataset 2: A subset of 497 apps from Dataset 1, for which source code is available in the F-Droid (Fdroid, 2025) repository. (3) Dataset 3: 700 apps obfuscated using various commonly-used obfuscators, derived from LibLoom. (4) Dataset 4: 721 apps obfuscated with different technologies, sourced from LibScan, which already includes all the app data provided by ORLIS and ATVHunter. (5) Dataset 5: 200 original apps containing widely-used vulnerable libraries, along with four different variants processed by R8, incorporating obfuscation and optimization.

Datasets 1 and 2 are utilized to address Q1, with Dataset 2 also employed in the real-world experiment for Q5. Furthermore, the ground-truth datasets (Datasets 3–5) are specifically designed to comprehensively address Q2, Q3, and Q4. The construction of Dataset 1 follows the following procedure: we selected apps based on the "Google Play Ranking" classification provided by AppBrain Appbrain (2025), which categorizes apps into 50 categories (such as finance, business, etc.). For each category, AppBrain provides a list of approximately 190 top-downloaded apps. Based on this list, we collected a total of 9,352 apps. For the remaining portion of Dataset 1, we randomly selected apps from Google Play with over 10,000 downloads and release dates between 2016 and 2024, consistent with the publication timeframe of Android TPL detection research, as summarized in Table 2. Please note that duplicate apps were excluded to ensure they were not counted multiple times. For Dataset 2, since F-Droid provides not only the source code but also precompiled apps (i.e., *APK*), we directly constructed our dataset using APKs obtained from F-Droid without making any modifications. In addition, similar to existing approaches such as LibHunter (Xie et al., 2024), we analyze each app's use of obfuscation and optimization, as well as the imported TPLs, by examining the configuration files included in their source code. Furthermore, Table 4 presents the obfuscation and optimization details for the three ground-truth datasets (Datasets 3–5). Notably, Dataset 3 employs more complex obfuscation techniques, making it ideal for evaluating a tool's resilience against obfuscation. Although Dataset 4 also underwent obfuscation, over 90% of its TPLs originate from the Android SDK (Android official libraries). This suggests that these standard libraries are either minimally affected by obfuscation or are not obfuscated at all. Therefore, Dataset 4 allows us to assess tool performance under low obfuscation intensity. Dataset 5, containing both original and R8-optimized apps, enables us to evaluate detection accuracy on original apps and measure resilience against optimization. The decision to use these datasets is based on the fact that the available ground-truth datasets – namely Dataset 3, Dataset 4, and Dataset 5 – comprehensively encompass original apps, obfuscated apps generated by leading obfuscators (e.g., Allatori, DashO), and R8-optimized apps, thereby ensuring sufficient representativeness. Since similarity-based approaches requires

Table 4
Datasets with ground-truth. (IR = identifier renaming, CS = code shrink, F = package flattening, R = repackaging, CFGR = control flow graph randomization, SE = string encryption, OBF/OPT Type = obfuscation and optimization technologies that the dataset claims to incorporate).

Dataset	tool	OBF/OPT Type	Subset	# of Apps
LibLoom (Dataset 3)	ProGuard	IR CS	PGD	100
		IR CS F	PGF	100
		IR CS R	PGR	100
	Allatori	IR CS CFGR R	ALR	200
	DashO	IR CS F CFGR SE	DOF	100
		IR CS R CFGR SE	DOR	100
LibScan (Dataset 4)	ProGuard	IR CS	PGD2	152
	Allatori	IR CS R	ALR2	188
	DashO	IR R	DOR2	159
		CFGR	DOC	79
		IR F	DOF2	79
		CS	DOS	79
LibHunter (Dataset 5)	R8	None	ORG	200
		IR	R8I	200
		CS	R8D	200
		CS OPT	R8O	200
		IR CS OPT	R8A	200

pre-extracted TPL feature profiles as reference targets, and we found that the datasets provided by LibLoom, LibScan, and LibHunter contain only a limited number of TPL samples. To construct a more comprehensive TPL feature database, we first integrated the TPLs from all three datasets, and then collected ten consecutive versions for each TPL with a specified version to generate their feature profiles. As a result, our databases comprises 5024 feature profiles. All our evaluation was conducted on a Linux Server equipped with Intel Xeon Gold 6338 CPUs and 256 GB RAM.

5.2. Q1: What is the prevalence of obfuscation and optimization in real-world closed-source apps?

Understanding the prevalence of obfuscation and optimization techniques in close-source apps helps evaluate the practical applicability of existing TPL detection tools and guides future research in this field. Since R8 is built upon ProGuard, the two share similar obfuscation rules, making it difficult to distinguish between them based solely on obfuscation features. Therefore, we classify ProGuard and R8 together under the label PR. To analyze obfuscation usage, we first categorize apps based on whether they are affected by PR tools (i.e., ProGuard or R8) or non-PR tools (e.g., Allatori or DashO). We then apply an optimization feature to differentiate between ProGuard and R8.

Based on the above description, we developed the following strategy to identify obfuscation and optimization in real-world Google Play apps (Dataset 1). First, we distinguish between obfuscated and original apps by analyzing whether identifiers such as package names and class names conform to obfuscation patterns proposed in existing studies (Wermke et al., 2018; Zhan et al., 2021a). Second, since ProGuard and R8 do not provide string encryption, whereas other obfuscators like DashO encrypt strings by default, we use string constant analysis to further differentiate obfuscators. If all string constants within an obfuscated package appear meaningless, we infer that the package has been processed by a non-PR tool. Otherwise, we attribute the obfuscation to ProGuard or R8. If an app is determined to be influenced by ProGuard or R8, we further assess whether advanced optimization has been applied. Our heuristic algorithm analyzes all methods within each class (Android standard class are skipped) to check for compliance with R8's method inlining and call-site optimization rules (Xie et al., 2024). If all methods within the app conform to these optimization rules, the app is classified into the fully optimized group (Full-OPT). During testing, we observed that some apps optimized only certain classes, and we categorize these apps into the partially optimized group (Part-OPT). Furthermore, All remaining apps are classified as non-optimized group (Non-OPT).

Based on our detection strategy applied to Dataset 1, the prevalence of obfuscation and optimization techniques in real-world apps is illustrated in Fig. 3. Our findings indicate that 42.2% (21,107) of the apps are original, while 57.8% (28,893) are obfuscated. This conclusion closely aligns with previous studies (Wermke et al., 2018). Although they did not report the proportion of optimized apps, they claimed that obfuscation is present in over 50% of popular apps (i.e., app with over 10 million downloads), and the more popular an app is, the higher the likelihood it has been obfuscated. More specifically, 44.5% (22,248) of the obfuscated apps appear to have been processed using ProGuard or R8, while 13.3% (6,645) contain encrypted strings, suggesting the use of non-PR tools. For the 22,248 apps influenced by ProGuard or R8, our analysis further reveals that: 29.4% (14,707) are fully optimized according to our detection rules, 4.3% (2,143) exhibit partial optimization, and 10.8% (5,398) are obfuscated but not optimized. To validate these findings, we manually examined Dataset 2 and compared its results with those from Dataset 1. Based on our detection strategy, Dataset 2 should contain: 248 original apps, 19 apps influenced by non-PR tools, 149 fully optimized apps, 21 partially optimized apps, 60 non-optimized apps. A detailed analysis of Dataset 2, conducted by comparing the source code of the released APKs and examining their obfuscation and optimization configurations in the `build.gradle` files, reveals the following: 248 apps did not enable any obfuscation or optimization options, while 19 apps employed string encryption and control flow graph randomization, indicating the use of non-PR tools. In addition, 151 apps were built using an Android Gradle Plugin version higher than 3.4.0, while 58 used a lower version. Further analysis of the configuration files confirmed that the 151 apps adopted R8 optimization, whereas the remaining 58 utilized ProGuard for obfuscation. This indicates that our approach misidentified only two obfuscated apps as false positives. The remaining 21 apps, despite enabling R8, disable optimization for TPLs via *-keep* rules in `build.gradle` files (e.g., *-keep class okhttp3.** means skipping optimization for all classes in the okhttp3 library). These results confirm that our strategy effectively distinguishes obfuscators and determines whether R8 optimization is enabled.

Compared with the findings reported in 2018, which indicated that approximately 50% of apps were obfuscated (Wermke et al., 2018), our experimental results based on Dataset 1 and Dataset 2 reveal a higher obfuscation and optimization rate of 57.8%. *This increase suggests that modern apps are increasingly adopting obfuscation techniques to protect intellectual property*. At the same time, *apps also tend to undergo*

Table 5
Comparison Results of Dataset 3.

Level	Data	LibScout			LibID			LibLoom			LibScan			LibHunter		
		PRC	REC	F1	PRC	REC	F1	PRC	REC	F1	PRC	REC	F1	PRC	REC	F1
Variety	PGD	82.2%	15.6%	26.2%	33.2%	48.6%	39.3%	90.2%	43.3%	58.5%	26.8%	2.8%	5.1%	6.5%	84.3%	12.0%
	PGF	85.2%	11.2%	19.8%	27.7%	36.5%	31.5%	79.3%	38.0%	51.4%	26.6%	2.7%	4.9%	6.5%	84.2%	12.0%
	PGR	88.7%	10.3%	15.9%	13.7%	42.2%	20.7%	85.8%	36.9%	51.6%	27.4%	2.8%	5.2%	6.5%	84.0%	12.0%
	ALR	51.5%	4.7%	8.7%	13.6%	61.2%	22.2%	82.3%	86.0%	84.1%	40.8%	11.6%	18.1%	5.2%	79.5%	9.7%
	DOF	36.4%	0.4%	0.9%	10.8%	48.6%	17.6%	85.6%	45.6%	59.5%	31.4%	3.5%	6.3%	4.6%	39.4%	8.2%
	DOR	41.7%	0.5%	1.1%	12.3%	3.1%	4.9%	82.3%	36.1%	50.1%	26.4%	3.1%	5.5%	4.5%	39.5%	8.1%
	TOT	73.4%	6.7%	12.3%	27.2%	36.5%	22.9%	**84.3%**	54.2%	**65.9%**	34.8%	5.5%	9.5%	5.6%	**70.1%**	10.4%
Version	PGD	74.6%	14.1%	23.8%	26.3%	38.5%	31.2%	79.3%	38.0%	51.4%	15.5%	1.6%	3.0%	4.3%	55.8%	8.0%
	PGF	79.1%	10.4%	19.4%	21.8%	28.6%	24.7%	66.6%	35.5%	46.3%	16.0%	1.6%	3.0%	4.2	55.0%	7.9%
	PGR	76.4%	8.9%	15.9%	10.3%	31.7%	15.5%	65.2%	28.1%	39.3%	17.9%	1.9%	3.4%	4.2%	54.8%	7.8%
	ALR	39.5%	3.6%	6.6%	10.8%	48.6%	17.6%	66.0%	68.8%	67.3%	28.1%	8.0%	12.4%	2.8%	43.3%	5.3%
	DOF	36.4%	0.4%	0.9%	10.4%	2.1%	3.5%	69.4%	36.9%	48.2%	27.5%	3.1%	5.5%	1.8%	15.7%	3.3%
	DOR	41.7%	0.5%	1.7%	9.7%	2.4%	3.9%	63.8%	28.0%	41.1%	26.4%	3.1%	5.5%	1.7%	15.4%	3.1%
	TOT	64.3%	5.9%	10.8%	13.1%	28.6%	18.0%	**67.3%**	**43.3%**	**52.7%**	24.6%	3.9%	6.7%	3.2%	40.4%	6.0%

code optimization prior to release. Notably, since such optimizations can sometimes interfere with the correct functioning of TPL code, developers may choose to disable these transformations specifically for TPLs. Nevertheless, optimization remains prevalent in closed-source apps, making it a critical factor that TPL detection tools must account for. Additionally, ProGuard and R8 are more commonly adopted than commercial obfuscators, likely due to their free availability and seamless integration with Android Studio, which significantly enhances their accessibility and usability. However, a notable number of apps still employ commercial obfuscators to achieve stronger protection. This underscores the necessity for TPL detection tools to incorporate resilience against the obfuscation techniques commonly used by both mainstream and commercial obfuscators. Moreover, while existing research frequently places disproportionate emphasis on the impact of obfuscation and optimization, it tends to overlook the performance of TPL detection tools on original apps. In reality, such apps account for 42.2% of the total, *underscoring the need for future work to include effectiveness on original apps as a fundamental evaluation criterion*.

> **Summary of Q1**: Obfuscation and optimization are widespread in close-source apps, with most developers utilizing ProGuard and R8 for these purposes. However, a significant number of apps still rely on commercial obfuscators to protect their intellectual property.

5.3. Q2: How effective are existing approaches?

To evaluate the effectiveness of the selected tools, namely LibScout, LibID, LibLoom, LibScan, and LibHunter, we first conducted experiments using three publicly available datasets and evaluated the tools' effectiveness in detecting TPL variety (Section 5.3.1). We then assessed the resilience of the core TPL detection techniques to obfuscation and optimization (Section 5.3.2), as well as their version identification capabilities (Section 5.3.3). Next, we analyzed the main factors contributing to the false positives and false negatives observed during the evaluation (Section 5.3.4).

5.3.1. Variety detection effectiveness

From the results of each comparison tool shown in Tables 5, 6, and 7, it is evident that the early works, LibScout and LibID, both exhibit relatively low performance across all three datasets. LibScout excels in precision, whereas LibID achieves a higher recall. LibScout's strength lies in its precision, while LibID demonstrates a higher recall. However, both tools heavily rely on package structures. For instance, LibScout's Merkle Tree feature is highly sensitive to the position of packages and classes within the app's code structure—any change can result in drastically different tree representations. Similarly, LibID's matching algorithm requires the package hierarchy to remain consistent before and after obfuscation. Since both obfuscators and optimizers can perform package-level transformations such as package flattening and repackaging, these dependencies significantly hinder the tools' effectiveness. As a result, both LibScout and LibID consistently demonstrate poor performance across all datasets. Specifically, LibScout achieves F1 scores of only 12.3%, 39.4%, and 42.8%, while LibID scores 22.9%, 41.4%, and 30.7% on the three datasets, respectively. The other two state-of-the-art tools, LibScan and LibHunter, appear to perform well primarily on their own provided datasets. LibScan achieves a high F1 score of 90.4% on its dataset (Dataset 4), but its performance declines sharply on Dataset 3 and Dataset 5, with only 6.7% and 22.0% F1 scores in TPL variety detection. Since Dataset 4 primarily includes official Android libraries, which are generally less affected by code transformations, LibScan performs well on this dataset. However, the more complex and diverse obfuscation techniques employed in Dataset 3 significantly hinder LibScan's ability to detect libraries accurately, as its strict matching rules – such as requiring set-inclusion relationships for call-chains and opcodes – are easily disrupted by such transformations. Moreover, while LibScan performs relatively well in detecting original apps in Dataset 5 (achieving a 70.6% F1 score), optimization significantly reduces feature integrity and disrupts the set-inclusion relationships of features due to techniques like code shrinking, method inlining, and call-site optimization. This severely limits LibScan's effectiveness in detecting optimized apps, leading to an overall F1 score of just 22.0% on Dataset 5. As for LibHunter, it achieves a good recall rate, but its high false positive rate substantially limits its F1 score performance. Similar to LibScan, it attains its best TPL variety detection rate (67.3%) on its own dataset (Dataset 5) but performs poorly on the other datasets, with only 6.0% and 11.9% F1 scores. One major limitation of LibHunter lies in its reliance on string constants—a feature highly vulnerable to various obfuscation techniques. To mitigate the impact of method inlining and call-site optimization, LibHunter employs advanced fuzzy features and a compensatory matching algorithm, both of which demonstrate notable effectiveness against R8 optimizations, as evidenced by the evaluation results. However, to account for the reduced feature set in optimized apps, LibHunter adopts a relatively low detection threshold (0.2). While this design enables it to identify optimized libraries effectively, it leads to suboptimal performance on both original and obfuscated apps—two categories that are also widely encountered in real-world apps.

LibLoom is the only tool that performs consistently well across all three datasets. Its F1 scores in Dataset 3 and Dataset 4 reach 65.9% and 91.4%, respectively, making it the best performer among all the compared tools. In Dataset 5, its overall performance is 63.1%, second only to LibHunter's 67.3%. LibLoom leverages the package hierarchy in conjunction with an entropy-based algorithm to segment feature units,

Table 6
Comparison Results of Dataset 4.

Level	Data	LibScout			LibID			LibLoom			LibScan			LibHunter		
		PRC	REC	F1	PRC	REC	F1	PRC	REC	F1	PRC	REC	F1	PRC	REC	F1
Variety	PGD2	78.9%	96.5%	86.3%	68.0%	97.3%	80.0%	95.6%	96.5%	96.1%	94.2%	96.7%	95.4%	6.7%	100%	12.7%
	ALR2	92.1%	9.1%	16.6%	62.5%	29.7%	40.3%	92.9%	95.5%	94.0%	92.6%	87.7%	90.1%	8.7%	92.3%	15.9%
	DOC	76.9%	4.1%	7.8%	2.6%	0.9%	1.3%	96.6%	97.4%	97.0%	96.5%	95.7%	96.1%	6.1%	97.0%	11.4%
	DOS	83.3%	4.1%	7.9%	3.4%	1.3%	1.9%	96.4%	89.8%	93.0%	94.0%	74.0%	82.8%	6.0%	97.0%	11.3%
	DOF2	80.0%	3.3%	6.0%	2.6%	0.4%	0.6%	87.6%	74.3%	80.4%	89.9%	77.8%	83.4%	6.0%	88.8%	11.2%
	DOR2	89.3%	18.0%	30.0%	85.6%	18.4%	30.3%	77.7%	96.3%	86.0%	92.1%	87.8%	89.9%	11.3%	86.6%	19.9%
	TOT	81.2%	26.0%	39.4%	59.3%	31.7%	41.4%	89.5%	**93.4%**	**91.4%**	**93.0%**	88.0%	90.4%	7.7%	92.8%	14.3%
Version	PGD2	73.2%	90.5%	92.1%	67.3%	96.4%	79.2%	94.1%	95.0%	94.6%	94.1%	95.0%	94.6%	6.5%	95.8%	12.1%
	ALR2	91.0%	9.0%	16.4%	60.4%	28.7%	38.9%	78.0%	80.0%	79.0%	89.8%	85.0%	87.4%	6.7%	65.6%	11.3%
	DOC	53.8%	2.9%	5.5%	1.3%	0.4%	0.6%	95.4%	96.2%	95.8%	96.6%	95.8%	96.2%	5.8%	92.3%	10.8%
	DOS	75.0%	3.7%	7.1%	3.4%	1.3%	1.9%	93.2%	86.8%	89.9%	94.0%	74.0%	82.8%	5.5%	89.8%	10.4%
	DOF2	60.0%	2.5%	4.8%	2.6%	0.4%	0.6%	83.3%	70.7%	76.5%	88.7%	76.8%	82.3%	5.5%	81.9%	10.3%
	DOR2	84.8%	17.2%	29.0%	84.2%	18.1%	29.8%	76.0%	94.3%	84.2%	91.2%	87.0%	89.1%	8.9%	68.4%	15.7%
	TOT	76.3%	24.4%	37.0%	58.2%	31.1%	40.6%	83.4%	**87.0%**	85.1%	**91.6%**	86.6%	**89.0%**	6.5%	77.6%	11.9%

enabling it to capture richer structural information while reducing susceptibility to package obfuscation. Building on this segmentation, LibLoom constructs a package filter using fuzzy class features, which effectively narrows down the matching scope by filtering out irrelevant package candidates. Notably, LibLoom achieves this without relying on fine-grained features such as opcodes, call-chains, or string constant—features that are more susceptible to code obfuscation and optimization, this design enhances its adaptability compared to other tools. Despite LibLoom's consistently strong performance across all three datasets, certain limitations remain. In particular, one of its core detection features – fuzzy method signatures – is susceptible to disruption from optimization techniques such as call-site optimization. Additionally, to ensure high detection precision, LibLoom employs a relatively strict matching threshold (0.6). While this strategy is effective under minimal code transformations, it becomes a liability in the context of aggressive code optimization, which can significantly compromise the integrity of extracted features. As a result, LibLoom performs poorly on optimized apps, achieving only a 13.6% F1 score, as shown in Table 7. Nonetheless, because Dataset 5 includes a combination of original, obfuscated, and optimized apps, LibLoom's strong performance on the former two categories conceals its shortcomings in handling optimization-induced changes.

Overall, these findings underscore a fundamental limitation of current TPL detection tools: their *inability to effectively handle diverse types of apps*. For example, approaches designed to resist the effects of optimization such as LibHunter often exhibit diminished detection performance on both original and obfuscated apps. Conversely, techniques tailored to counter common obfuscation strategies tend to lack robustness against aggressive optimizations. We argue that this is because different *obfuscation and optimization techniques affect existing detection methods in distinct ways*, making it highly challenging for any single approach to effectively handle them simultaneously. Given the widespread adoption of these techniques in real-world apps and their disruptive impact on TPL detection, it is crucial to examine how they influence the core technologies employed by state-of-the-art tools. This investigation serves to validate our hypothesis. A detailed analysis is presented in Section 5.3.2.

5.3.2. Obfuscation- and optimization-resilience

Although the ground-truth experimental results demonstrate that obfuscation and optimization can negatively impact detection outcomes, the final results alone do not clearly reveal how these transformations affect specific detection techniques. This is because the final effectiveness of each tool is closely related to various factors, such as the developer's coding proficiency and specific implementation details. Therefore, we conducted a more in-depth evaluation focusing on commonly used unit segmentation methods, detection features, and matching strategies, in order to assess the resilience of these specific techniques to obfuscation and optimization. Just as fuzzy features are designed to remain consistent before and after transformation – enabling reliable comparisons between obfuscated and non-obfuscated code – other features and strategies should also demonstrate high stability under such transformations. High consistency indicates minimal impact from obfuscation and optimization, whereas significant deviations suggest a higher susceptibility to these techniques. Specifically, we analyze the robustness of commonly used unit segmentation approaches based on package hierarchies, as well as LibLoom's entropy-based segmentation technique, in the presence of package obfuscation. In addition, we evaluate the reliability of key detection features widely adopted in TPL detection – namely fuzzy method signatures, fuzzy class features, opcode sequences, call-chains, and string constants – under various obfuscation and optimization scenarios. We also examine the robustness of common matching strategies, such as set-inclusion relationships of opcode and call-chain. Our evaluation is conducted under the transformations applied by widely used tools such as DashO, ProGuard, and R8 to assess whether these detection technologies can maintain consistency and reliability. Through this analysis, we aim to gain deeper insights into how obfuscation and optimization affect TPL detection technologies, ultimately offering valuable insights for the development of more robust TPL detection approaches.

Our experiments evaluating key TPL detection technologies under obfuscation and optimization scenarios utilized the 10 most popular TPLs from the original apps in Dataset 1 as study targets, as they represent the most common usage scenarios in real-world apps. For each TPL, we identified apps from Datasets 2–5 that incorporate it and collected their original, DashO-obfuscated, ProGuard-obfuscated, and R8-optimized versions. The purpose of extracting these versions was to enable direct comparisons between the original apps and their transformed variants (e.g., original vs. ProGuard-obfuscated), thereby facilitating an analysis of how widely used TPL detection techniques are affected by obfuscation and optimization. Since the extent of certain obfuscation techniques, such as code shrinking, depends on how much TPL code is actually used by the app, we adopted a statistical averaging approach to reduce bias. Specifically, for each popular TPL, we collected 300 apps that incorporate it, along with four corresponding transformed variants (i.e., original, obfuscated and optimized), for subsequent comparative analysis. If our datasets did not yield a sufficient number of apps incorporating a given popular TPL (e.g., *Jsoup*, which was used by only 141 apps in our dataset), we supplemented the data by searching repositories such as GitHub (Github, 2025b) for additional apps that include the target TPL, and constructed their original, obfuscated, and optimized variants accordingly.

After determining the analysis data, we designed the following experiments to evaluate the performance of existing key TPL detection techniques under obfuscation and optimization. First, we calculated the number of original library packages that retained their structural integrity after undergoing package flattening and repackaging introduced

Table 7
Comparison Results of Dataset 5.

Level	Data	LibScout			LibID			LibLoom			LibScan			LibHunter		
		PRC	REC	F1	PRC	REC	F1	PRC	REC	F1	PRC	REC	F1	PRC	REC	F1
Variety	ORG	80.5%	85.6%	83.0%	73.4%	88.2%	80.1%	79.4%	90.3%	84.4%	84.5%	61.1%	70.6%	47.0%	91.0%	62.0%
	R8I	98.0%	11.2%	20.1%	88.9%	9.7%	17.5%	85.5%	61.6%	71.6%	100%	1.7%	3.3%	72.1%	81.0%	76.3%
	R8D	89.6%	44.4%	59.4%	78.9%	4.6%	8.7%	80.3%	90.3%	85.0%	100%	1.7%	3.3%	45.8%	91.0%	60.9%
	R8O	90.0%	2.1%	4.1%	100%	2.6%	5.0%	94.0%	7.3%	13.6%	100%	1.5%	2.9%	79.1%	64.4%	71.0%
	R8A	90.0%	2.1%	4.1%	100%	2.6%	5.0%	94.0%	7.3%	13.6%	100%	1.5%	2.9%	79.1%	66.4%	72.2%
	TOT	84.7%	28.6%	42.8%	76.2%	19.2%	30.7%	**81.8%**	51.4%	63.1%	66.7%	13.1%	22.0%	58.8%	**78.8%**	**67.3%**
Version	ORG	56.4%	60.0%	58.1%	58.0%	69.7%	63.3%	66.0%	95.0%	74.9%	70.2%	53.0%	61.3%	39.4%	81.0%	53.0%
	R8I	34.7%	3.9%	7.1%	44.2%	4.8%	8.7%	64.9%	46.7%	54.3%	100%	1.7%	3.3%	54.5%	61.3%	57.7%
	R8D	62.7%	31.1%	41.6%	78.9%	4.6%	8.7%	65.8%	74.0%	69.6%	100%	1.7%	3.3%	37.4%	74.9%	50.1%
	R8O	70.0%	1.6%	3.2%	80%	2.0%	4.0%	75.0%	5.8%	10.9%	100%	1.5%	2.9%	44.2%	36.0%	39.7%
	R8A	70.0%	1.6%	3.2%	80%	2.0%	4.0%	75.0%	5.8%	10.9%	100%	1.5%	2.9%	43.4%	36.5%	39.7%
	TOT	57.2%	19.3%	28.9%	58.5%	14.8%	23.5%	**66.7%**	41.5%	**51.0%**	58.5%	11.5%	19.3%	42.5%	**57.0%**	48.7%

Table 8
The impact of default DashO obfuscation (DSO) on widely-used features in the 10 most popular TPLs (The package hierarchy similarity was omitted due to its negligible value (< 1%) before and after package obfuscation. P/R = The proportion of cases in which the LibLoom's entropy-based strategy correctly identifies the destination package after package flattening or repackaging obfuscation, M = The proportion of method retained in app after obfuscation and optimization, C = proportion of class retained in app after obfuscation and optimization, CS = similarity of fuzzy class feature in app before and after obfuscation or optimization, OF = the proportion of the method's opcode that adheres to the set-inclusion relation after obfuscation or optimization, OS = opcodes similarity, CF = the proportion of the method's call-chain that adheres to the set-inclusion relation, Cc = call-chain similarity, SC = string constant similarity).

TPLs	DSO_P	DSO_R	DSO_M	DSO_C	DSO_CS	DSO_OF	DSO_OS	DSO_CF	DSO_Cc	DSO_SC
Commons	96.8%	95.7%	89.1%	92.3%	84.0%	92.7%	75.6%	99.7%	87.1%	1.0%
Glide	98.8%	98.6%	87.0%	88.1%	85.4%	93.7%	70.0%	100%	94.0%	6.9%
Gson	96.4%	97.2%	88.7%	90.3%	76.1%	93.2%	70.8%	100%	90.6%	0%
Guava	97.3%	97.5%	90.4%	86.3%	76.9%	100%	71.4%	100%	97.0%	13.0%
HttpC	95.8%	96.2%	86.8%	84.3%	90.0%	81.3%	78.9%	95.2%	99.7%	0%
Jackson	96.3%	96.9%	95.0%	92.8%	88.9%	94.0%	73.8%	100%	93.9%	6.4%
Jsoup	96.7%	96.8%	83.4%	82.4%	73.2%	80.1%	64.4%	90.1%	81.0%	5.4%
Log4j	98.2%	98.6%	84.6%	85.5%	88.4%	83.2%	66.3%	100%	84.5%	0%
Okhttp	95.3%	95.5%	91.7%	92.9%	83.5%	91.3%	72.5%	98.8%	83.7%	0%
Retrofit	96.7%	96.3%	96.1%	96.3%	93.3%	82.4%	79.9%	94.6%	85.7%	0%
AVG.	96.8%	96.9%	89.3%	89.1%	83.4%	89.2%	72.4%	97.8%	89.7%	3.3%

by ProGuard, DashO, and R8. We then assessed whether LibLoom's entropy-based segmentation strategy could accurately identify the destination package in apps subjected to such transformations. Next, we analyze the persistence of fuzzy method signatures and fuzzy class features following obfuscation and optimization. These features serve as fundamental components in TPL detection research and are widely adopted by tools such as LibScout, LibID, and others. Subsequently, we evaluate whether method opcodes and call-chains preserve the set-inclusion relationship proposed by LibScan after obfuscation and optimization, and measure the similarity of opcodes and call-chains between the original apps and their variants. Finally, we quantify the extent to which string constants, a key feature identified by LibHunter, are affected by obfuscation and optimization. Based on experiments conducted 300 aforementioned apps and averaged accordingly, the results for DashO, ProGuard, and R8 are presented in Tables 8 9, and 10, respectively.

As shown in Table 8 and its caption, DashO significantly alters the package hierarchy, with the similarity between the original and obfuscated structures dropping below 1%. The entropy-based algorithm proposed by LibLoom accurately identifies nearly 97% of destination packages under DashO's package flattening and repackaging obfuscation, demonstrating its robustness and effectiveness. Furthermore, DashO obfuscation applies a moderate level of code shrinking, preserving 89.3% of methods and 89.1% of classes. Although fuzzy class features retain a high similarity post-obfuscation (83.4%), we observe that class fingerprints still undergo some degree of alteration. Consequently, approaches such as LibScan, which depend on identical class fingerprints for successful matching, may fail to identify certain classes, thereby compromising their overall effectiveness. Additionally, not all method opcode features satisfy the set-inclusion relation, where original method opcodes should form a subset of the obfuscated ones. Each library exhibits some deviation from this relation to varying extents. Notably, for *Retrofit*, only 79.9% of methods adhere to this constraint. This discrepancy occurs because, although DashO's control-flow randomization introduces new conditional statements, its code shrinking mechanism removes unused code and simplifies syntactic structures. The combined effect of these techniques compromises the reliability of the set-inclusion relationship. Moreover, the statistical results indicate that most call-chain features (97.8%) satisfy the set-inclusion relation. For instance, libraries such as *Gson* exhibit a 100% compliance rate with this relation. Besides, the similarity between call-chain features before and after obfuscation is often close to 90%. Although DashO can introduce junk method calls, these calls typically involve repeated invocations of existing methods and thus have minimal impact on the set-inclusion relation. Since DashO employs string constant encryption, these values are rarely preserved in their original form after obfuscation—on average, only 3.3% of strings remain unchanged.

Table 9 presents the impact of ProGuard's default obfuscation and optimization on representative TPL detection features. First, although ProGuard's package obfuscation still thoroughly disrupts the package hierarchy—posing a significant challenge to unit segmentation approaches that rely on structural information. LibLoom's entropy-based algorithm maintains strong performance, achieving 97.3% and 97.5% accuracy on ProGuard-processed apps, demonstrating its robustness. Unlike commercial obfuscators such as DashO, ProGuard is more aggressive in code shrinking, retaining only 48.6% of methods and 62.2% of classes after processing. Some libraries, such as *Log4J*, preserve as little as 10% of their original methods and classes. Due to the elimination of characteristics such as methods from the class, the similarity of

Table 9
The average impact of default ProGuard obfuscation (PGR) on TPL detection features in the 10 most popular TPLs, the package hierarchy similarity was also excluded due to its extremely low value.

TPLs	PGR_P	PGR_R	PGR_M	PGR_C	PGR_CS	PGR_OF	PGR_OS	PGR_CF	PGR_Cc	PGR_SC
Commons	97.4%	97.9%	33.2%	46.0%	68.9%	85.7%	96.5%	99.3%	98.4%	99.4%
Glide	96.1%	96.3%	69.1%	83.2%	76.5%	81.3%	96.5%	92.4%	98.7%	99.3%
Gson	98.3%	98.5%	58.6%	76.5%	64.5%	88.9%	89.5%	99.3%	91.3%	91.4%
Guava	98.4%	98.2%	20.6%	22.4%	60.2%	69.5%	97.0%	99.7%	99.1%	99.9%
HttpC	96.8%	97.3%	49.8%	60.6%	70.0%	91.3%	94.1%	95.2%	99.7%	99.7%
databind	96.5%	97.3%	52.9%	74.7%	72.1%	87.1%	95.4%	97.1%	96.8%	99.7%
Jsoup	97.7%	97.6%	55.7%	76.7%	70.4%	91.0%	95.4%	97.7%	99.1%	99.9%
Log4j	96.4%	96.5%	5.7%	7.3%	59.8%	82.0%	96.0%	99.8%	98.5%	99.9%
Okhttp	97.9%	98.0%	69.9%	86.4%	75.4%	86.5%	95.8%	98.4%	98.9%	99.3%
Retrofit	97.6%	97.7%	70.8%	88.4%	78.0%	84.9%	97.5%	98.6%	99.0%	99.2%
AVG.	97.3%	97.5%	48.6%	62.2%	69.6%	84.8%	95.4%	97.8%	98.0%	98.8%

fuzzy class features between the same class before and after reduction decreases to approximately 70%, and the reduced level of similarity may lead to the inability of the class to successfully match. Similar to the impact exerted by DashO, ProGuard is capable of altering the set-inclusion relationship of opcode features, only 84.8% of the opcode features can fit this relationship after ProGuard processing. Compared to DashO, which significantly impacts opcode similarity, ProGuard allows opcode features to maintain a higher level of consistency (95.4%). This indicates that ProGuard modifies opcode features to a lesser extent than DashO. Additionally, 97.8% of call-chain features generally satisfy the set-inclusion relation and maintain a high similarity rate (98.0%) after ProGuard processing, further demonstrating the reliability of call-chain features. In addition, ProGuard does not alter string constant features, as most libraries preserve 98.8% string constant similarity under its influence.

Building upon ProGuard's obfuscation and optimization, R8 further transforms code characteristics, exerting an even greater impact on feature integrity through various advanced optimization techniques. For instance, R8 can perform method inlining and class merging, thereby significantly impacting the integrity of methods and classes. As shown in Table 10, consistent with previous experimental results, the entropy-based algorithm designed to counteract package obfuscation maintains a high level of accuracy, approaching 95%. However, as previously mentioned, R8 optimization significantly compromises feature integrity. After optimization, only about 23.5% of the original methods are retained. On average, classes preserve 43.6% of their original count and exhibit 53.6% similarity in structure post-optimization. Moreover, R8 frequently disrupts the set-inclusion relation of opcodes and call-chains, reducing the effectiveness of these rules in such scenarios. In ProGuard-obfuscated libraries, 84.8% of method opcodes and 97.8% of method call-chains maintain the set-inclusion relationship. However, in R8-optimized libraries, these figures drop further to 72.2% and 78.0%, respectively. This decline is primarily due to method inlining, which integrates opcodes and method calls from multiple methods, thereby affecting opcode and call-chain similarity. As a result, after R8 optimization, opcode and call-chain similarity decrease to 83.6% and 78.3%, respectively. Furthermore, while R8 does not encrypt string constants, method inlining can merge string constants from multiple methods, leading to a slight decrease in string constant similarity, averaging 89.6%.

In summary, while DashO has minimal impact on the number of methods and classes, it alters the opcode set and disrupts the set-inclusion relationship by introducing additional conditional structures and junk method calls. Additionally, its encryption algorithms significantly affect string constant feature. Consequently, approaches that primarily rely on these features, such as LibHunter, may be severely hindered by DashO obfuscation. ProGuard's code shrinking techniques, on the other hand, substantially reduce the number of methods and classes, exerting an even greater impact on feature integrity than DashO obfuscation, and this reduction can impair all TPL detection approaches. Furthermore, its code shrinking can disrupt the set-inclusion relationship of opcodes. However, ProGuard's impact on overall opcode similarity, call-chains, and string constant remains relatively limited. Notably, R8 adopts a more aggressive approach by removing unused code while significantly altering code characteristics. Although its impact on opcode similarity and string constant similarity is less pronounced than that of DashO, its optimization techniques exert a more substantial influence on other critical TPL detection features. In addition, the impact of R8 optimization on call-chain features remains relatively minor, indicating that call-chain features can consistently maintain their reliability despite various external influences.

From the above summary, it is evident that existing features exhibit limitations in effectively handling all external influences. More critically, as previously discussed, *obfuscation and optimization affect app features to varying degrees*, particularly in terms of feature integrity retention, which differs significantly among original, obfuscated, and optimized apps. This variability makes it challenging for current research to balance an appropriate detection strategy that performs well across all scenarios. For example, under original or lightly obfuscated conditions, feature integrity remains consistently high, allowing most feature units to be successfully matched. In such cases, detection tools typically adopt a high threshold (e.g., 0.9, representing the ratio of matched feature units to total library feature units) to ensure accurate library identification while minimizing false positives. However, when an app undergoes ProGuard's shrinking process, this threshold must be lowered (e.g., LibLoom reduces it to 0.6) to compensate for the substantial reduction in retained feature units. For apps subjected to more aggressive code optimization, the threshold must be further relaxed to maintain detection performance. Therefore, LibHunter requires a low matching threshold of 0.2 to detect optimized libraries. This is because the maximum number of possible matched methods in optimized apps is typically only about 25.7% of that in the original state. While a lower threshold enhances sensitivity in TPL detection, it also increases susceptibility to false positives. This issue is particularly pronounced when analyzing original apps, as the higher feature integrity in such cases significantly elevates the likelihood of false positives. Consequently, *existing features struggle to accurately detect original, obfuscated, and optimized apps simultaneously*, even though all of these types are highly prevalent in real-world scenarios. This inherent trade-off underscores the difficulty of designing robust TPL detection mechanisms that can effectively accommodate diverse code transformations.

5.3.3. Version identification capability

Accurate identification of library versions is crucial, as it serves as a fundamental prerequisite for uncovering 1-day vulnerabilities, detecting malicious functionalities, and addressing various security-related issues. LibScout is the first tool claiming to distinguish different TPL versions, it demonstrates high precision in version identification. However, due to its low recall, the F1 scores for version detection remain significantly low, at 10.8%, 37.0%, and 28.9% across the three datasets. While LibID adopts finer-grained features, such as opcodes derived from control-flow graph, its version detection performance

Table 10
The average impact of default R8 optimizations (OPT) on TPL detection features in the 10 most popular TPLs, the package hierarchy similarity was also excluded due to its extremely low value.

TPLs	PGR_P	PGR_R	OPT_M	OPT_C	OPT_CS	OPT_OF	OPT_OS	OPT_CF	OPT_Cc	OPT_SC
Commons	95.2%	95.8%	17.3%	33.8%	48.1%	71.4%	86.0%	72.5%	81.0%	93.1%
Glide	94.2%	94.5%	26.8%	44.3%	51.9%	72.9%	82.5%	86.1%	81.8%	89.9%
Gson	96.4%	96.7%	28.8%	50.7%	55.1%	82.7%	80.3%	78.5%	71.8%	88.3%
Guava	96.3%	96.0%	9.7%	12.5%	59.1%	63.7%	84.1%	99.7%	72.4%	81.4%
HttpC	94.4%	95.0%	34.2%	39.6%	50.3%	81.3%	84.8%	79.3%	83.0%	86.9%
databind	94.4%	95.2%	25.5%	58.7%	53.0%	75.6%	83.4%	62.7%	76.3%	92.0%
Jsoup	95.8%	95.6%	27.7%	59.4%	61.4%	74.8%	79.4%	81.7%	84.4%	93.9%
Log4j	94.4%	94.5%	1.7%	5.3%	32.1%	57.9%	84.8%	76.6%	81.4%	98.6%
Okhttp	96.2%	96.3%	36.2%	61.3%	59.6%	76.7%	85.7%	70.0%	77.4%	88.6%
Retrofit	95.5%	95.6%	49.3%	70.6%	64.8%	65.2%	85.0%	72.4%	73.2%	83.4%
AVG.	95.3%	95.5%	25.7%	43.6%	53.5%	72.2%	83.6%	78.0%	78.3%	89.6%

does not surpass that of LibScout. This limitation arises primarily because the BIP algorithm employed by LibID for feature matching relies heavily on the package structure, which is susceptible to modifications through obfuscation. Such changes can result in a substantial number of incorrect matches, thereby adversely affecting accurate version identification. On the other hand, LibScan achieved an impressive F1 score of 89% for TPL version detection within its own dataset, which highlights its strong performance in identifying Android libraries with minimal obfuscation. Nevertheless, this performance drops drastically in other datasets, where it records F1 scores of only 6.7% and 19.3%. Although LibScan incorporates features such as opcodes, which are more robust for identifying different library versions, opcodes are highly susceptible to obfuscation and optimization, thereby adversely affecting LibScan's ability to accurately detect TPL versions. The latest approach, LibHunter, extends the use of advanced fuzzy features and introduces a compensation mechanism to alleviate the impact of code optimizations performed by R8. Although this strategy enhances feature generalization, version detection inherently relies on features that are more specific and distinctive. Consequently, this trade-off undermines LibHunter's ability to achieve high detection precision. Its version detection precision reaches only 6%, 11.9%, and 48.7% across the evaluated datasets. Even on its own dataset, LibHunter's version identification capability remains inferior to that of LibLoom. Surprisingly, LibLoom continues to demonstrate strong performance in version identification, achieving the highest F1 scores in Dataset 3 and Dataset 5, as well as the highest recall in Dataset 4. As a tool that leverages coarse-grained class features as its fundamental feature for library identification, LibLoom's robust performance can be attributed to two primary reasons. First, library updates often involve substantial changes, which frequently manifest as modifications to class structures and attributes, making LibLoom's feature effective for distinguishing versions. Second, version identification relies on the accurate detection of TPL variety, a task in which LibLoom excels. The improved accuracy in identifying TPL categories provides LibLoom with greater opportunities to correctly determine the corresponding library versions.

Although existing similarity-based approaches claim the ability to differentiate between various library versions, Table 2 reveals substantial variation in the granularity of features employed across different studies. For instance, LibScout primarily relies on fuzzy method signatures for detection, whereas LibScan performs deeper static analysis by incorporating opcode sequences and call-chain features. However, prior research has not thoroughly investigated whether these features inherently capture the nuanced changes introduced throughout the evolution of TPLs. More critically, the superior performance of the coarse-grained features adopted by LibLoom – surpassing those based on finer-grained representations – raises an essential question: *To what extent do different types of detection features vary in their ability to distinguish between TPL versions*? This issue can be investigated by evaluating the extent to which these features, under ideal conditions (i.e., in the absence of obfuscation, optimization, or other external influences), are capable of uniquely identifying different library versions. Such an evaluation provides a clearer assessment of the optimal discriminative power of existing detection features for version identification—if distinct versions cannot be differentiated even under ideal circumstances, it is highly unlikely that they can be accurately identified in the presence of obfuscation and optimization. To this end, we conducted a comprehensive experiment evaluating whether the representative features employed by state-of-the-art open-source tools—including fuzzy method signatures (LibScout), fuzzy class features (LibLoom), fuzzy class features combined with opcodes of CFG (LibID), fuzzy class features with opcodes and call-chains (LibScan), and fuzzy class features with opcodes and string constants (LibHunter)—can effectively capture characteristic differences between adjacent versions of widely used libraries. For consistency, we selected the top 10 most popular TPLs, as previously described, and performed pairwise comparisons between each library's adjacent versions. A feature was considered effective if it could identify at least one distinguishing element between any two versions, thereby confirming its ability to detect version updates. Conversely, if no such distinction was found, the feature was deemed ineffective for version differentiation. We then recorded the number of version pairs each feature set could successfully distinguish across the 10 selected TPLs. The results, as presented in Table 11, offer valuable evidence of the discriminative power of different detection features with respect to version changes.

From Table 11, it is evident that as features become more precise and diversified in complexity, the number of distinguishable versions gradually increases. Surprisingly, despite being constructed without requiring static analysis, fuzzy method signatures (FMS) and fuzzy class features (FCF) exhibit strong capabilities for version differentiation. Notably, FCF achieves complete identification of different versions of *OkHttpClient*, *Jsoup*, and *Log4j*, while also performing comparable effectiveness to more granular features such as fuzzy class features with opcodes and call-chains (FCOC) and fuzzy class features with opcodes and string constant (FCOS) in distinguishing versions of *Gson*, *Glide*, and *Retrofit*. However, certain libraries, such as *OkHttp3* and *Jackson-Databind*, introduce only minor changes across some version updates. In such cases, only more detailed and complex features such as opcodes, call-chains, and string constants are capable of effectively capturing the underlying differences. Among the advanced feature sets, opcodes combined with string constants (95.6%) outperform opcodes combined with call-chains (92.4%). This is likely because libraries often store their version numbers as strings, making string constants highly sensitive to version differences. However, this feature is less reliable in obfuscated scenarios, as string encryption techniques can completely alter string constants.

Another noteworthy observation is that, despite multiple iterations in TPL detection feature development, even the most advanced techniques struggle to differentiate certain TPL versions, such as *Glide*, *Gson*, *Guava*, and *Retrofit*. This challenge arises because TPL detection relies on a large set of library feature profiles for comparison, necessitating highly efficient feature selection. Consequently, existing features primarily capture abstract control flow and data flow information, which

Table 11
Popular obfuscation changes (FMS = fuzzy method signatures; FCF = fuzzy class features; FCO = fuzzy class features with opcodes; FCOC = fuzzy class features with opcodes and call-chains; FCOS = fuzzy class features with opcodes and string constant; Versions Counts = the number of library versions that a feature can distinguish; Neither = the number of versions that are indistinguishable from all features).

TPLs	FMS	FCF	FCO	FCOC	FCOS	Versions Counts	Neither
Commons	93.9%	96.9%	100%	100%	100%	32 33 34 34 34	0
Glide	82.8%	86.2%	86.2%	86.2%	86.2%	29 30 30 30 30	4
Gson	91.7%	91.7%	91.7%	91.7%	91.7%	36 36 36 36 36	3
Guava	92.3%	92.3%	97.4%	97.4%	97.4%	37 37 39 39 39	1
OkHttpClient	100%	100%	100%	100%	100%	38 38 38 38 38	0
Jackson-Databind[a]	70.5%	73%	77.3%	83.4%	100%	15 19 26 36 63	0
Jsoup	100%	100%	100%	100%	100%	48 48 48 48 48	0
Log4j	100%	100%	100%	100%	100%	52 52 52 52 52	0
Okhttp3	76.7%	76.7%	81.7%	90.0%	100%	46 46 49 54 60	0
Retrofit2	81%	81%	81%	85.7%	81%	20 20 20 21 20	3
AVG.	88.2%	88.9%	91.1%	92.4%	95.6%	–	11

[a] For the Jackson-Databind row, we have truncated the Version Counts to two digits, with the actual values requiring the addition of 100 to the displayed figures.

limits their ability to detect all types of code changes. For instance, some TPL version updates only modify logical relationships, which do not affect call-chain or string constant features. Additionally, the opcodes corresponding to modified statements may already be present elsewhere within the function body. As a result, even if the library is updated, the overall opcode composition may remain unchanged, making it difficult to differentiate between versions using current feature sets. If these indistinguishable versions contain security vulnerabilities, they could pose significant risks to the security of the software supply chain.

In summary, *even the most sophisticated features cannot reliably distinguish all library versions*. Based on this premise, coarse-grained features that do not rely on static analysis – such as fuzzy method signatures and fuzzy class features – are well-suited for large-scale detection due to their high efficiency. In contrast, fine-grained features (e.g., opcodes and call chains) provide greater discriminative power for distinguishing between different library versions, particularly when multiple fine-grained features are combined. However, this improved accuracy comes at the cost of significantly higher computational overhead (see Section 5.4 for details). Moreover, in many cases, fine-grained features yield only marginal improvements in version identification accuracy. This limitation stems from the fact that most TPL detection features are intentionally designed to ensure acceptable comparison efficiency, enabling the rapid filtering of similar libraries from large-scale TPL feature database. However, this design trade-off inherently limits their ability to capture the subtle semantic changes introduced by library version updates. Therefore, when time cost is not a limiting factor, accurately identifying specific library versions can be more reliably accomplished through a two-stage process. The first stage uses TPL detection tools to narrow down potential libraries and their versions, while the second stage conducts control-flow and data-flow analyses to uncover version-specific characteristics, such as those revealed by patch presence test techniques (Zhang and Qian, 2018).

5.3.4. False positive and false negative

Analyzing the causes of false positive and false negative provides valuable insights into the limitations of existing TPL detection approaches and can guide the development of more robust methods. Fig. 4 presents the False Positive Rate (FPR) and False Negative Rate (FNR) produced by various tools across different datasets. Here, FPR denotes the ratio of false positives to the total number of detected libraries, while FNR reflects the ratio of missed true positives to the actual number of ground-truth libraries. As shown in the figure, LibLoom performs relatively well compared to other tools. Although it maintains a low FPR, its FNR is notably high—approaching 50% in both Dataset 3 and Dataset 5. This is primarily due to LibLoom's strict matching strategy and relatively high detection threshold. LibLoom proceeds

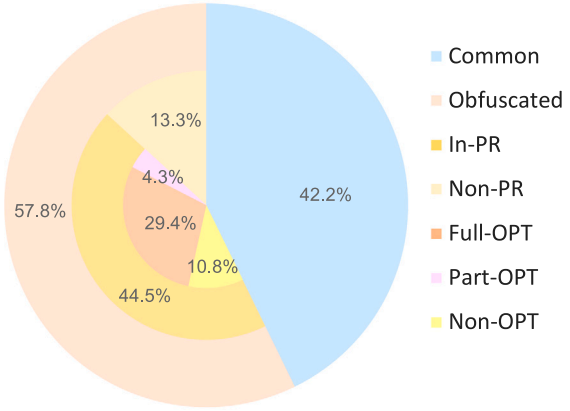


Fig. 3. The result of pervasiveness of obfuscation and optimization in Google Play apps.

with further comparisons only after the package similarity between the TPL and the app reaches 0.8, and it considers a TPL present in the app only if more than 60% of the library's class features are similar to the app's class features. Furthermore, its core feature, the fuzzy method signature, is sensitive to changes introduced by call-site optimizations, which reduces its ability to detect TPLs in highly optimized apps. Another state-of-the-art tool, LibHunter, achieves a lower false negative rate than LibLoom. However, it exhibits high FPR across all three datasets—94.4%, 92.3%, and 41.2%, respectively. This can be attributed to three main factors: First, LibHunter adopts highly abstracted features, replacing not only obfuscation-prone identifiers with the placeholder "X", but also optimization-sensitive identifiers with another placeholder "?". This abstraction reduces feature specificity, increasing the likelihood of incorrect matches. Second, the compensation mechanism in its compensatory greedy matching process may further contribute to erroneous feature pairings. Lastly, to counteract aggressive code shrinking caused by optimizations, LibHunter uses a relatively low matching threshold, which, while reducing false negatives, results in a higher number of false positives. As for the remaining tools, most suffer from an inability to simultaneously maintain low false positive and false negative rates. For instance, tools such as LibScout and LibScan generally achieve lower false positive rates, primarily due to their stricter feature definitions or matching rules, including the use of Merkle Tree and set-inclusion relationships of call-chain. However, this strictness also results in higher FNR, indicating a trade-off between precision and recall. In contrast, LibID exhibits a different

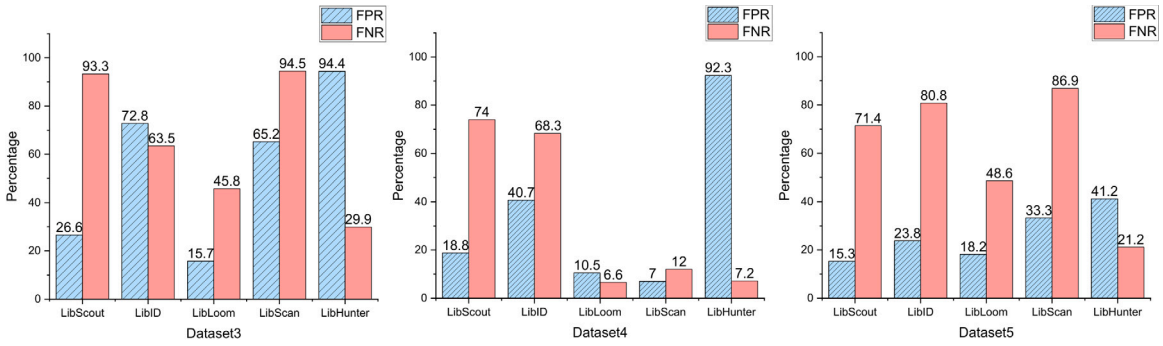


Fig. 4. False positive rate and false negative rate for each detection tool across different datasets.

Table 12
Statistical analysis of attributes in high false-positive frequency libraries.

TPL	Packages	Classes	Methods	Code Lines
Threetenabp	1.0	2.0	3.2	53.5
GlideImageViewFactory	2.0	2.0	3.0	47.0
Butterknife	1.7	18.6	33.8	620.1
Coil	2.1	21.5	38.6	1221.6
K2pdfopt	1.0	3.0	1.0	36.8
Initializer	1.9	2.0	5.2	47.9
Ext-strikethrough	1.0	3.0	8.0	69.0
Exoplayer	1.0	1.0	1.0	6.0
Zxing	1.0	4.0	6.3	143.0
Djvulibre	1.0	7.7	2.0	121.5
Average	14.2	325.7	1289.6	30 685.3

issue: both its false positives and false negatives remain at relatively high levels, suggesting limitations in both the accuracy and coverage of its detection strategy.

The above analysis indicates that obfuscation, optimization, and improperly set thresholds, whether excessively high or low, are among the primary causes of both false negatives and false positives. Obfuscation and optimization can significantly reduce feature completeness and disrupt established matching rules (e.g., set-inclusion relationship of call-chain). Furthermore, an improperly set threshold can also lead to inaccurate detection results. If the threshold is set too low, features from unrelated TPLs may easily satisfy the matching conditions, leading to false positives. On the other hand, if the threshold is too high, it may result in a higher number of false negatives. In addition to the reasons mentioned above, our further analysis reveals another key contributor to false positives: the inherent characteristics of certain TPLs. We examined the top 10 TPLs with the highest false positive frequencies in our evaluation and collected statistics on their number of packages, as well as their total number of classes, methods, and lines of code. These metrics, which form the basis of many TPL detection features, are strong indicators of a library's feature complexity. We also computed the average values of these metrics across all TPLs for comparison, all the aforementioned data are presented in Table 12. Notably, TPLs that are frequently misclassified typically exhibit a very simple structure: they contain fewer packages, classes, and methods, and minimal code lines. For example, different versions of the *threetenabp* library on average include only one packages, two classes, and 3.2 methods, with just 53.5 lines of code—figures that are substantially lower than the overall averages of 14.2 packages, 325.7 classes, 1,289.6 methods, and 30,685.3 lines of code. This functional simplicity limits the TPL's ability to generate rich and distinctive features, thereby increasing the likelihood of incorrect matches with unrelated features and making it easier to meet the matching thresholds. Together, these factors ultimately lead to false positives.

Table 13
Quartiles of TPL detection time, PA refer to the average time cost in preprocessing.

Metrics	LibScout	LibID	LibLoom	LibScan	LibHunter
PA	65.7 s	164.5 s	81.9 s	175.4 s	416.3 s
Q1	4.5 s	55.5 s	6.6 s	107.0 s	216.0 s
Mean	10.2 s	1711.8 s	32.6 s	135.7 s	451.7 s
Median	8.9 s	274.4 s	11.5 s	119.0 s	342.5 s
Q3	13.0 s	1655.4 s	31.2 s	149.2 s	606.3 s

Summary of Q2: (1) No single tool demonstrates consistently superior performance across original, obfuscated, and optimized apps, this is primarily because (2) both obfuscation and optimization can significantly impact the integrity of most existing techniques, resulting in substantial differences in feature representation and completeness among these three types of apps. (3) Even the most sophisticated features are still unable to reliably distinguish all library versions. (4) Obfuscation, optimization, and predefined thresholds are the major contributors to false positives and false negatives. In addition, overly simplistic TPL features are another key cause of false negatives.

5.4. Q3: What is the efficiency of current approaches?

The efficiency of TPL detection is a crucial metric for evaluating the scalability of detection tools, as these tools often need to analyze large numbers of apps within a limited timeframe. For example, the *Facebook SDK* vulnerability affected millions of open-source components, compelling security professionals to perform large-scale TPL detection under tight time constraints. This highlights the critical importance of detection efficiency. Typically, the similarity-based TPL detection process comprises two main phases (all of our selected tools follow this approach). The first phase involves preprocessing and feature extraction (referred to as the preprocessing stage). During this stage, both the target app and the TPLs are decompiled, and relevant features are extracted from the resulting intermediate representations. The second phase is the matching stage, in which similarity computations are performed between the feature of the target app and those in the TPL database to determine the final detection results. Existing research primarily focuses on the overhead from the second phase, overlooking the impact of preprocessing stage. To provide a more comprehensive assessment, our study not only records the average time consumption for preprocessing (PA) but also evaluates the feature comparison efficiency of each tool by reporting key statistical metrics – including Q1, mean, median, and Q3 values – as commonly done in prior research. The detailed results are presented in Table 13.

Firstly, we observe that LibScout achieves the highest efficiency, requiring only 65.7 s on average to complete the preprocessing stage

for an app and 10.2 s for feature comparison. LibScout extracts only fuzzy method signatures, and its process of constructing a Merkle Tree is straightforward. Additionally, during the feature comparison process, even when features undergo minor changes, the Merkle Tree can effectively detect these variations and terminate unnecessary matching early. These factors collectively contribute to its superior performance in terms of efficiency. Following closely behind is LibLoom, which takes an average of 114.5 s (81.9 s for preprocessing and 32.6 s for comparison) per app. LibLoom's efficiency is attributed to two primary factors: (1) it extracts only fuzzy class features, avoiding fine-grained feature extraction processes such as opcodes, call-chains, and string constants, which are computationally expensive. This simplification streamlines the preprocessing process and enhances feature matching efficiency; and (2) it employs a package filter strategy that discards invalid matches by evaluating the overlap rate of package features, effectively eliminating most irrelevant TPLs before the class matching process. Starting with LibScan, there is a noticeable decline in TPL detection efficiency because subsequent tools rely on fine-grained static analysis to generate features and employ more complex matching strategies or algorithms. LibScan achieves an average detection time of 311.1 s (135.7 s for preprocessing and 175.4 s for feature matching), which represents an acceptable computational overhead. The tool's efficiency is attributed to its requirement that matched features be either completely identical or satisfy set-inclusion relationships, enabling a relatively rapid matching process. In contrast, the state-of-the-art tool, LibHunter, requires a significantly longer average detection time of 451.7 s for a single app. Notably, its preprocessing phase alone takes 416.3 s, and the entire detection process can take nearly 15 min per app. This imposes clear limitations when scaling to large datasets or high-throughput detection scenarios. The increased time cost of LibHunter stems from its reliance on advanced fuzzy features and the adoption of a compensation mechanism. The former requires additional expensive static analysis, while the latter demands matching more features, both of which substantially increase computational overhead. As for LibID, since it relies on the BIP algorithm, which is an NP-hard problem, the processing time grows exponentially as app complexity increases or when a large number of TPLs need to be compared. In our experiments, while the preprocessing phase took only 164.5 s per app, the comparison phase required 1711.8 s. This significant time overhead makes LibID nearly impractical for large-scale TPL detection.

> **Summary of Q3**: The efficiency differences among various tools are substantial, primarily due to the different types of features they employ and their feature matching strategies or algorithms. Extracting features through fine-grained static analysis can greatly increase detection time, while computationally intensive algorithms may also be unsuitable for the TPL detection field, which demands large-scale feature matching.

5.5. Q4: How generalizable are existing TPL detection tools when applied to diverse scenarios?

The generalizability of TPL detection tools reflects their effectiveness across diverse scenarios and serves as a key indicator of their practical applicability. We assess the generalizability of each tool from two perspectives: (1) performance fluctuations across the three ground-truth datasets, and (2) discrepancies between our evaluation results and those reported in the original publications, including both effectiveness and efficiency differences. For the first aspect, we plotted the performance curves of each evaluated tool across different datasets, as shown in Fig. 5. For the second part, we compared the effectiveness and efficiency of LibLoom on Dataset 3 with the results reported in its original paper, LibScan on Dataset 4 with its published statistics, and LibHunter on Dataset 5 with the corresponding data from its original study. Among these, we found that our TPL variety detection F1 score for LibLoom (65.9%) was even slightly higher than the result reported in its original paper (65.5%). It is worth noting that LibLoom did not report TPL version identification results in its publication, and thus, a direct comparison with our version identification results is not possible. Similarly, LibScan's performance exhibited only a minor degradation of approximately 6% compared to its reported values. Specifically, its original paper reported a 97.4% accuracy in variety detection and 95.0% in version identification, while our experiments on Dataset 4 yielded scores of 90.4% and 89.0%, respectively. In contrast, only LibHunter exhibited a significant performance discrepancy, with an F1 score difference exceeding 16%. To more clearly highlight the significant differences between our results and those reported in the LibHunter paper, we include both our evaluation results and the results from the original publication (denoted as LibHunter') in Fig. 6. Additionally, we observed that the detection times of both LibLoom and LibScan in our evaluation were significantly higher compared to the results reported in their original publications. Specifically, LibLoom's per-app detection time increased from 8.9 s to 32.6 s, while LibScan's rose from 46.0 s to 135.7 s. Please note that we are unable to compare the efficiency of LibHunter, as its original paper does not report concrete timing data. In the following, we provide a detailed analysis and explanation of the experimental results.

(1) Based on the performance curves in Fig. 5 and our previous analysis, we conclude the following: both LibScout and LibID consistently exhibit poor performance; LibScan and LibHunter perform well only on their respective original datasets; and only LibLoom maintains strong performance across all datasets. However, even LibLoom is only effective for original and obfuscated apps, and performs poorly when faced with code optimization. The performance fluctuations observed in these tools can be attributed to two primary factors. First, *each dataset contains different types of mutated apps*. Dataset 3 consists of heavily obfuscated apps, Dataset 4 includes apps with relatively mild obfuscation, and Dataset 5 primarily consists of optimized apps. The variations in feature expression and integrity across these app types pose substantial challenges to existing tools, which generally *lack the versatility to handle all categories effectively*. This limitation contributes to their inconsistent performance across different datasets. Second, *the implementation frameworks employed by the tools also have a considerable impact on their effectiveness*. Tools such as LibID, LibScan, and LibHunter depend on the Androguard framework, which requires converting TPL *JAR* files into *DEX* format using intermediary tools like dex2jar. However, due to semantic and structural differences between Java bytecode and Android bytecode, this conversion process is often lossy—critical code features or structural elements may be lost or altered during conversion, which reduces feature accuracy. Additionally, our experiments reveal that the version of the decompilation tool used can further influence the results: outdated versions of the tool may fail to decompile newer bytecode instructions or recognize updated Android API, which further hampers the tool's ability to extract consistent and accurate features.

(2) Although under optimization scenarios (R8O and R8 A), our evaluation results closely align with those reported by LibHunter, we observed significant discrepancies in the results obtained from testing original (ORG) and R8-obfuscated (R8I and R8D) apps compared to LibHunter's original paper. Specifically, the differences are approximately 30.4%, 16.4%, and 24.8%, respectively. We attribute the performance discrepancy of LibHunter to three main factors: (1) LibHunter employs an overly low detection threshold to accommodate optimized apps that retain only a small proportion of features. However, since original, obfuscated, and optimized apps exhibit substantial differences in feature preservation, this design leads to a higher likelihood of false positives in the original and obfuscated datasets where feature retention is relatively high. Specifically, LibHunter adopts a threshold of 0.2 (in comparison, LibLoom uses a threshold of 0.6), meaning that a TPL is reported once only 20% of its features are matched. Such a low threshold inevitably increases false positives, as only a

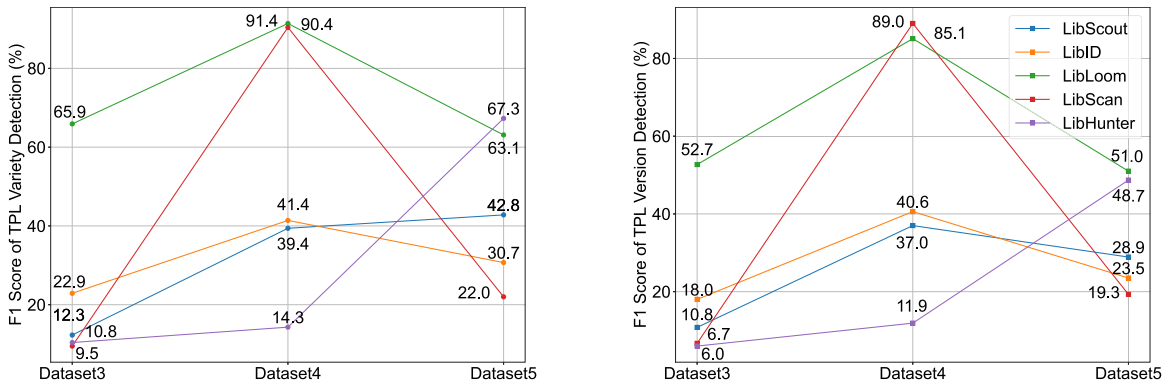


Fig. 5. The performance curves of various TPL detection tools across different datasets.

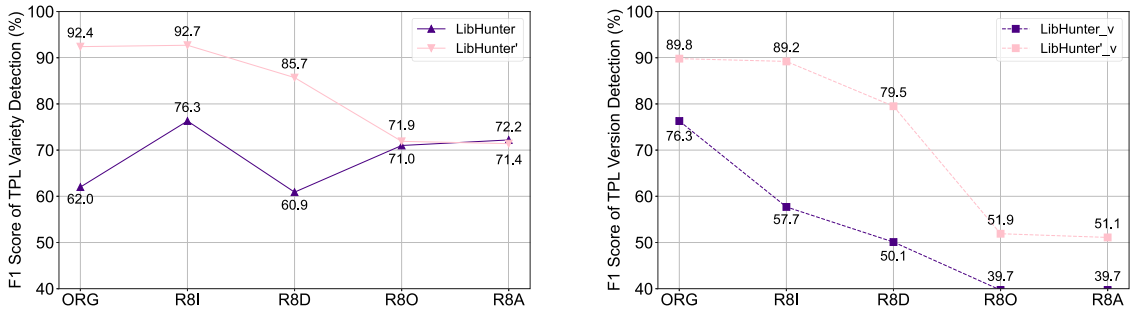


Fig. 6. The performance discrepancy between our results and the results reported in the LibHunter's paper.

small number of matched features suffices to trigger a TPL report. In addition, LibHunter incorporates fuzzy features, which further increase the similarity among features and make it easier for the matched proportion to exceed the threshold, thereby amplifying the risk of false positives. (2) The number of TPL profiles used in the original LibHunter evaluation is limited. Specifically, LibHunter only provides 177 TPLs, which happen to cover all libraries present in the apps from its own dataset (Dataset 5). A smaller reference set may help reduce false positives caused by LibHunter's low threshold and compensation mechanism. However, our study simulates a more realistic detection scenario by employing a comprehensive TPL feature library with 5024 profiles. This expanded reference set likely exacerbates LibHunter's susceptibility to false positives, thereby impacting its overall performance. (3) Differences in the experimental environment may also contribute to the observed discrepancies. Since LibHunter does not explicitly specify its original test environment, our evaluation testbed may differ from the conditions under which the tool performs optimally, potentially leading to subpar results on certain metrics. Moreover, we attribute the efficiency differences between our results and those reported in LibLoom and LibScan to similar underlying reasons. For instance, LibLoom's dataset includes only 551 distinct TPLs, which is significantly fewer than the 5024 TPLs used in our study. This substantial increase in the reference set means that each app must be compared against a much larger number of TPLs, inevitably increasing the overall detection time.

Summary of Q4: Most TPL detection tools demonstrate strong performance only when evaluated on their own datasets, largely because each approach is particularly effective at detecting specific types of apps (e.g., original apps). In addition, differences in the TPL feature databases may also influence overall detection performance.

5.6. Q5: How do current approaches perform on real-world apps?

Although the tools' performance has been evaluated in previous sections using three ground-truth datasets, these datasets inevitably involve some degree of manual modification, such as artificially applied obfuscation and optimization. As a result, their performance on these datasets may not fully reflect their effectiveness in real-world scenarios. To address this limitation, we use Dataset 2 as a real-world benchmark to evaluate the selected tools, including LibScout, LibID, LibLoom, LibScan, and LibHunter, and to assess their effectiveness in detecting TPL varieties and identifying library versions under realistic conditions. The results are presented in Fig. 7.

As shown in Fig. 7, the average detection performance of the tools on real-world apps improves compared to their performance on the ground-truth datasets. As a result, the performance differences among the tools are significantly smaller than those observed on the ground-truth datasets. For example, in the comparison results on Dataset 3, LibLoom outperformed LibScout by more than a factor of five in TPL category detection. However, in the real-world setting, this gap narrows drastically to just 6.5%. This phenomenon can be attributed to the diverse release practices in real-world scenarios, where apps may be released in their original form, after obfuscation, or following optimization. Therefore, TPL detection tools must possess the capability to handle all three types of app distributions. Nevertheless, as our previous analysis suggests, no existing tool is capable of effectively handling all three scenarios. Instead, each tool demonstrates specific strengths and weaknesses. For instance, LibLoom performs well on original and obfuscated apps but struggles with optimized apps, whereas LibHunter excels in detecting optimized apps but exhibits a high false positive rate on original ones. As a result, the complementary strengths and limitations of each tool tend to balance out, leading to more comparable performance under real-world conditions.

It is worth noting that although the performance gap among TPL detection tools in category identification is relatively small, typically within a few percentage points, their capabilities in version identification differ more significantly. LibLoom continues to be the most

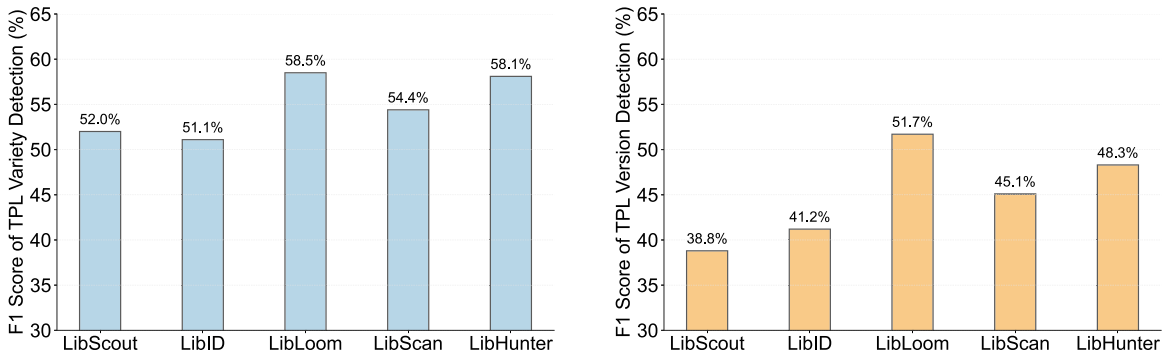


Fig. 7. The real-world performance of existing approaches.

effective tool in this task, achieving an F1 score of 51.7%, followed by LibHunter with 48.3%. LibScout, on the other hand, shows weaker performance, with an F1 score of only 38.8%. This discrepancy reflects the varying effectiveness of different feature designs in handling fine-grained version detection. LibScout relies primarily on fuzzy method signatures to distinguish between versions, which proves inadequate since library updates often do not involve changes at the method level. In contrast, LibLoom employs fuzzy class features combined with package filter techniques to improve version identification accuracy. LibHunter also enhances version differentiation by comparing opcode features and string constants, enabling more precise identification of library versions. Accordingly, we suggest that future studies, provided efficiency constraints are met, explore the use of finer-grained features and composite feature strategies to improve the precision of library version identification.

We also compared the detection time of each tool in the real-world experiments, defined as the sum of preprocessing time and feature matching time. We observed that all approaches exhibited increased detection times when applied to real-world apps. This is primarily because real-world apps are significantly larger—on average, nearly twice the size of those apps in the ground-truth datasets. Consequently, more time is required for app analysis, and a greater number of features must be matched. Among all the tools evaluated, LibScout remained the most efficient, with its average detection time increasing only slightly from 74.6 s to 91.9 s. This efficiency can be attributed to its use of a Merkle Tree structure, which enables fast and effective feature matching. LibLoom and LibScan also experienced moderate increases in detection time (from 114.5 s to 145.1 s for LibLoom, and from 311.1 s to 389.1 s for LibScan), both benefiting from the use of Bloom Filters to abstract features, which enhances their feature matching efficiency. In contrast, LibHunter and LibID exhibited significantly higher detection time increases. LibHunter's average detection time rose from 868.0 s on the ground-truth datasets to 1250.4 s in real-world scenarios. This increase is due to its reliance on fine-grained static analysis to extract advanced fuzzy features for resilience against code optimization, resulting in longer preprocessing times. Additionally, its compensation mechanisms require extended matching time. As for LibID, the inherent inefficiency of its BIP algorithm constrains its overall performance in large-scale real-world detection tasks.

Summary of Q5: Existing approaches demonstrate relatively strong performance in real-world scenarios, generally surpassing their results on ground-truth datasets. Although the differences in TPL variety detection capabilities among tools are relatively minor, the disparities in their version identification performance remain substantial. Moreover, the detection time required by current tools increases significantly when analyzing real-world apps compared to ground-truth data.

6. A integrated tool based on study insights

Based on the identified strengths and limitations of existing TPL detection approaches in the previous sections, we aim to enhance overall detection performance by integrating their complementary techniques, particularly for type versions of apps (i.e., original, obfuscated, and optimized), while striving to minimize time overhead. Following the common practice adopted by most prior work, we employed a two-stage analysis framework to balance efficiency and accuracy. In the first stage, certain features such as LibRadar's API calls and LibScout's fuzzy method signatures based on Merkle Trees were excluded because they are not robust against obfuscation and optimization, and provide limited support for version identification. In contrast, given the strong performance of LibLoom's fuzzy class feature across Dataset 3, 4 and 5, we adopted it as our coarse-grained detection feature. This feature is used to preliminarily determine whether a TPL is likely to be present in the target app—for each TPL class, we identify its potential matching candidates in the app based on fuzzy class features for subsequent analysis. Unlike LibScan, which requires the two classes to have identical fingerprints for candidate selection, our approach does not depend on strict fingerprint equality. This design choice is motivated by our findings in Section 5.3.2, which demonstrate that fuzzy class features are susceptible to changes caused by obfuscation and optimization. Instead, we use the Jaccard index (Wikipedia, 2025b) to measure the similarity between the fuzzy class features of app classes and TPL classes. If the similarity exceeds a predefined threshold θ_1, the two classes are considered potential matches. Since a single TPL class may correspond to multiple potential matches in the app, we only select the app candidate with the highest similarity to the TPL class and store the result in a mapping (e.g., <app class, TPL class>). The mapping are then used as input for the next stage of analysis.

In the second stage, it is critical to improve version identification accuracy by leveraging multiple fine-grained features (as demonstrated in Section 5.3.3), while maintaining robustness against both obfuscation and optimization. Based on the analysis presented in Sections 5.3.1 and 5.3.2, we observed that the call-chain feature performs consistently well across original, obfuscated, and optimized apps. Therefore, it was included in our fine-grained feature set. We do not consider the set-inclusion relationship of call-chains, as done in LibScan, because it is highly susceptible to disruptions caused by R8 optimization. Since LibHunter is the only tool that demonstrates resilience to R8 optimization, we not only adopted its features (i.e., opcodes and string constants) along with the call-chain to construct method features, but also incorporated its fine-grained static analysis strategy to enhance the optimization resilience of our features. Then, we perform method matching between each TPL class and its corresponding candidate classes (i.e., the output from the previous stage), and compute the number of matched methods between them. Considering that string encryption can completely alter string constants, rendering this feature ineffective under obfuscation, we adopt a robust method matching

Table 14
Comparison on the F1-score of our tool and its peer tool across Dataset 3, 4 and 5.

Level	Dataset	TCT	LibScout	LibID	LibLoom	LibScan	LibHunter
Library	Dataset 3	**76.2%**	12.3%	22.9%	65.9%	9.5%	10.4%
	Dataset 4	**96.7%**	39.4%	41.4%	91.4%	90.4%	14.3%
	Dataset 5	**77.9%**	42.8%	30.7%	63.1%	22.0%	67.3%
	Total	**83.6%**	31.5%	31.7%	73.5%	40.6%	30.6%
Version	Dataset 3	**66.3%**	10.8%	18.0%	52.7%	6.7%	6.0%
	Dataset 4	**93.7%**	37.0%	40.6%	85.1%	89.0%	11.9%
	Dataset 5	**60.6%**	28.9%	23.5%	51.0%	19.3%	48.7%
	Total	**73.6%**	25.6%	27.4%	62.9%	38.3%	22.2%

strategy to mitigate this limitation. Specifically, two methods are considered a match if the Jaccard similarity of their features exceeds a threshold θ_2, based on either of the following conditions: (1) opcodes and call-chain features, or (2) those plus string constants. This strategy enables the matching process to fall back on opcode and call-chain feature comparison under string encryption obfuscation, while leveraging string constants to improve accuracy when processing original or optimized apps. After the matching process, we compute the ratio of matched methods to the total number of methods in the TPL. If this ratio exceeds a certain threshold θ_3, the presence of the TPL is confirmed, and the TPL version with the highest number of matched methods is selected as the reported version. To improve comparison efficiency, we implemented an *early termination mechanism* during the method matching process: if the number of TPL methods that fail to find a similar app method for pairing reaches $(1-\theta_3)\times total_tpl_methods$, the matching process is terminated early to avoid unnecessary computation.

We implemented the proposed feature set and matching strategy in a new TPL detection tool named **TCT** (i.e., **T**PL detection tools with **C**ombined **T**echnologies). To tune the matching thresholds of TCT, we randomly selected 100 apps from Dataset 2 and finalized the thresholds θ_1, θ_2 and θ_3 as 0.7, 0.8, and 0.3, respectively. We then evaluated TCT on Datasets 3, 4, and 5, and compared its performance against other state-of-the-art tools. The results are summarized in Table 14. According to the data, TCT achieves TPL variety detection accuracies of 76.2%, 96.7%, and 77.9% on the three datasets, with an overall accuracy of 83.6%. Its version identification accuracies reach 66.3%, 93.7%, and 60.6%, yielding an overall accuracy of 73.6%—all outperforming the other tools in comparison. Even when compared with the best-performing tool on the full test dataset – LibLoom – TCT consistently achieves a 10.1% improvement in TPL variety accuracy and a 10.7% increase in version detection precision, demonstrating its effectiveness in handling diverse types of apps and distinguishing subtle differences between library versions. These findings suggest that combining the strengths of existing features can significantly enhance TPL detection performance. Furthermore, we evaluated the average time overhead of TCT during the preprocessing and feature matching stages. The results show that TCT incurs a slightly higher preprocessing cost than LibHunter – 418.1 s versus 416.3 s – primarily due to its combination of multiple detection features and the use of LibHunter's fine-grained static analysis to improve feature resilience against optimization. However, thanks to our early termination mechanism, TCT exhibits significantly lower overhead during the feature matching stage compared to LibHunter, requiring only 219.7 s. Considering that TCT maintains robust performance across different types of apps, this level of time overhead is considered acceptable.

7. Discussion

Building on the preceding analysis and evaluation, this section summarizes lessons learned and offers insights to inform future research.

7.1. Lessons learned

Although TPL detection has been an active research area for over a decade, significant challenges remain. Our analysis reveals two key limitations in current TPL detection approaches: (1) their limited effectiveness across different types of apps (as demonstrated by the results of Q2 and Q4), and (2) their inability to comprehensively identify all library versions (as reflected in Q2 and Q5). In addition to the widely discussed factors such as obfuscation and optimization, we argue that these shortcomings are also closely related to the incomplete observations made by researchers. When detection methodologies are developed based on limited datasets without rigorous empirical validation, there is a high risk of drawing overgeneralized conclusions. We highlight the following misleading conclusions corrected by our study:

- **LibID** operates under the assumption that obfuscators do not modify the package structure and designs its detection mechanism accordingly. However, obfuscators frequently alter package hierarchies, rendering LibID's detection rules ineffective for obfuscated apps.
- **LibScan** assumes that method opcodes remain a superset of their original versions even after transformations such as obfuscation. However, both obfuscation and optimization can disrupt this set-inclusion relationship, leading to a significant decline in LibScan's detection performance.
- **LibHunter** concludes that commercial obfuscators are rarely used in practice, as its experiments on 2,347 open-source apps found that fewer than 10% employed commercial obfuscators. However, our analysis of 50,000 Google Play apps (99% of which are closed-source) reveals that commercial obfuscators is in fact much more prevalent. The limited scale of LibHunter's dataset and its reliance on conclusions drawn solely from open-source apps – which typically have little incentive to protect their already publicly available code – are likely the main reasons for this biased conclusion.

To enhance the robustness and generalizability of TPL detection tools and to draw more reliable conclusions, researchers should validate their underlying assumptions through comprehensive experimental studies. To this end, this work provides a detailed data-driven analysis. Through empirical evaluations and synthesis of the results, this work corrects misleading conclusions drawn by prior studies and provides statistically grounded support for future research.

We also observe that all cutting-edge open-source tools (i.e., LibScout, LibID, LibLoom, LibScan, and LibHunter) lack clear module decomposition, comprehensive API documentation, and extension guidelines, which may hinder users and developers seeking to adopt or extend these tools. To examine this issue in depth, we conducted a usability evaluation of aforementioned tools. Given that such evaluations often rely on human intuition and perception, which are susceptible to bias, we opted to utilize Large Language Models (LLMs) to perform the assessment in place of human reviewers. This choice is motivated by the nature of our evaluation task, which demands robust capabilities in semantic understanding, reasoning, analytical thinking, and subjective judgment—areas where LLMs have demonstrated particular strength. Recent studies have highlighted their impressive performance across a wide range of natural language processing tasks, including semantic understanding (Zhao et al., 2023c; Bubeck et al., 2023), reasoning and analysis (Xi et al., 2025; Lemieux et al., 2023), and evaluative assessment (Wu et al., 2025). The results indicate that recently released tools, such as LibScan, LibLoom, and LibHunter, demonstrate notable improvements in usability compared to earlier tools like LibScout and LibID. Nevertheless, further efforts are needed to simplify their usage and enhance the clarity of their outputs (Please refer to the Supplementary Material for detailed prompts and evaluation results). This suggests that authors of top-tier papers may sometimes overlook

the fact that their work often serves as a benchmark in the field, exerting a profound and lasting influence on practitioners. Therefore, we strongly recommend that future authors offer more comprehensive code documentation and extension guidelines. Doing so would greatly facilitate the adoption and enhancement of these tools by practitioners, ultimately contributing to the continued advancement and innovation in this domain.

7.2. Insights for future research

We now present our insights for future research, focusing on three critical aspects: selecting stable features, weighting key features, and refining comparison algorithms.

Selecting Stable Features. As analyzed in Section 5.3.3, coarse-grained fuzzy class features and fine-grained call-chain features exhibit relatively strong stability under varying conditions compared to other feature types. Fuzzy class features are effective in identifying potential class-matching candidates while reducing false positives, whereas call-chain features enhance the accuracy of match verification. For tools primarily designed to detect libraries in original and optimized apps, string constants can serve as reliable features due to their relative stability in these scenarios. In addition, future work can build upon the analysis of existing approaches to design more effective and robust features.

Weighting Key Features. In Section 5, we have highlighted that the primary challenge preventing current tools from effectively handling original, obfuscated, and optimized apps simultaneously lies in the substantial variation in feature completeness across these app types. However, existing research often treats all features equally, failing to account for the varying importance of different features. Unlike the unpredictable nature of code shrinking, certain optimization techniques can be anticipated in advance. For instance, static analysis can identify methods that remain unaffected by call-site optimization. By assigning higher confidence levels to these methods when they are matched through a weighted approach, the likelihood of detecting TPLs can be enhanced even in cases where feature completeness is significantly reduced. Moreover, assigning higher matching weights to structurally complex features, similar to the strategies employed by LibPecker and LibRoad, can also improve detection accuracy. For instance, classes with a large number of methods may be weighted more heavily, as complex features are less likely to result in false matches. As a result, successful matches of such features significantly increase confidence in the presence of TPLs.

Refining Comparison Algorithms. Due to the potential high similarity among different feature units, the feature matching process may associate multiple app feature units with a single TPL feature unit or vice versa, potentially resulting in incorrect matches. To address this, tools such as LibScan and LibHunter employ more fine-grained, iterative matching processes. While these methods help reduce mismatches, they do not fully eliminate them, as their reliance on greedy matching remains overly simplistic and fails to consider factors beyond feature similarity. In contrast, LibID introduces a Binary Integer Programming algorithm that incorporates class dependency information to enhance matching accuracy. However, due to constraints in both practical effectiveness and computational efficiency, its approach remains challenging to apply in real-world TPL detection scenarios. Therefore, optimizing existing matching algorithms to balance accuracy and efficiency presents a critical and valuable direction for future research.

8. Conclusion

In this paper, we conducted a comprehensive analysis and summary of existing TPL detection techniques. To validate our observations, we also performed large-scale experiments. Through these extensive experiments, we first examined the prevalence of original, obfuscated, and optimized applications to identify the external factors that TPL detection tools must account for. We then systematically evaluated the effectiveness of existing approaches in TPL variety detection and version identification, analyzed the impact of obfuscation and optimization on these detection technologies, and investigated the causes of false positives and false negatives. Furthermore, we tested the generalizability of the tools and validated their practical performance using real-world applications. Finally, we construct a new tool by integrating the strengths of existing feature designs, and summarize the lessons learned while providing insights to guide future research.

CRediT authorship contribution statement

Lige Zhan: Writing – review & editing, Writing – original draft, Software, Methodology, Data curation. **Jiang Ming:** Writing – review & editing, Conceptualization. **Jianming Fu:** Supervision, Funding acquisition. **Guojun Peng:** Supervision. **Letian Sha:** Writing – review & editing. **Lili Lan:** Investigation, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

We sincerely thank the reviewers for their valuable feedback and guidance in improving this paper. This work was sponsored by the National Natural Science Foundation of China (No. 62272351, No. 62172308, and No. 62172144). Jiang Ming was supported by National Science Foundation grants 2312185 and 2417055, as well as a Google Research Scholar Award.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.cose.2025.104672.

Data availability

Data will be made available on request.

References

Akram, Junaid, Shi, Zhendong, Mumtaz, Majid, Luo, Ping, 2018. DroidCC: A scalable clone detection approach for android applications to detect similarity at source code level. In: Proceedings of the 42nd Annual Computer Software and Applications Conference. COMPSAC '18.

Allatori, Accessed: 2025-08-31. Allatori Java Obfuscator, https://allatori.com/ [online].

Androguard, Accessed: 2025-08-31. Androguard, https://github.com/androguard/androguard [online].

Apache, Accessed: 2025-08-31. MVN Repository, https://mvnrepository.com/ [online].

Apktool, Accessed: 2025-08-31. Apktool, https://apktool.org/ [online].

Appbrain, Accessed: 2025-08-31. Appbrain, https://www.appbrain.com/apps/most-downloaded [online].

Backes, Michael, Bugiel, Sven, Derr, Erik, 2016. Reliable Third-Party Library Detection in Android and its Security Applications. In: Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security. CCS '16.

Bhoraskar, Ravi, Han, Seungyeop, Jeon, Jinseong, Azim, Tanzirul, Chen, Shuo, Jung, Jaeyeon, Nath, Suman, Wang, Rui, Wetherall, David, 2014. Brahmastra: Driving apps to test the security of third-party components. In: Proceedings of the 23rd USENIX Security Symposium. USENIX Security '14.

Bubeck, Sébastien, Chadrasekaran, Varun, Eldan, Ronen, Gehrke, Johannes, Horvitz, Eric, Kamar, Ece, Lee, Peter, Lee, Yin Tat, Li, Yuanzhi, Lundberg, Scott, et al., 2023. Sparks of artificial general intelligence: Early experiments with gpt-4. arXiv preprint arXiv:2303.12712.

Chen, Sen, Fan, Lingling, Chen, Chunyang, Xue, Minhui, Liu, Yang, Xu, Lihua, 2019. GUI-Squatting Attack: Automated Generation of Android Phishing Apps. IEEE Trans. Dependable Secur. Comput..

Chen, Kai, Liu, Peng, Zhang, Yingjun, 2014. Achieving accuracy and scalability simultaneously in detecting application clones on android markets. In: Proceedings of the 36th International Conference on Software Engineering. ICSE '14.

Chen, Kai, Wang, Xueqiang, Chen, Yi, Wang, Peng, Lee, Yeonjoon, Wang, XiaoFeng, Ma, Bin, Wang, Aohui, Zhang, Yingjun, Zou, Wei, 2016. Following devil's footprints: Cross-platform analysis of potentially harmful libraries on android and iOS. In: Proceedings of the 37th IEEE Symposium on Security and Privacy. S&P '16.

Chen, Sen, Xue, Minhui, Fan, Lingling, Hao, Shuang, Xu, Lihua, Zhu, Haojin, Li, Bo, 2018. Automated poisoning attacks and defenses in malware detection systems: An adversarial machine learning approach. Comput. Secur..

clarivate, Accessed: 2025-08-31. Journal Citation Reports, https://clarivate.com/academia-government/scientific-and-academic-research/research-funding-analytics/journal-citation-reports/ [online].

ClaudiuGeorgiu, Accessed: 2025-08-31. Obfuscapk, https://github.com/ClaudiuGeorgiu/Obfuscapk/ [online].

Crussell, Jonathan, Gibler, Clint, Chen, Hao, 2012. Attack of the Clones: Detecting Cloned Applications on Android Markets. In: Proceedings of the 17th European Symposium on Research in Computer Security. ESORICS '12.

Crussell, Jonathan, Gibler, Clint, Chen, Hao, 2014. Andarwin: Scalable detection of android application clones based on semantics. IEEE Trans. Mob. Comput..

CSRankings, Accessed: 2025-08-31. CSRankings: Computer Science Rankings, https://csrankings.org/#/index?all&us [online].

Cui, Huajun, Meng, Guozhu, Li, Yueqi, Li, Yuejun, Zhang, Yan, Sun, Jiyan, Zhu, Dali, Wang, Weiping, 2022. LibHunter: An unsupervised approach for third-party library detection without prior knowledge. In: Proceedings of 2022 IEEE Symposium on Computers and Communications. ISCC '22.

Dai, Jiarun, Zhang, Yuan, Jiang, Zheyue, Zhou, Yingtian, Chen, Junyan, Xing, Xinyu, Zhang, Xiaohan, Tan, Xin, Yang, Min, Yang, Zhemin, 2020. BScout: Direct whole patch presence test for java executables. In: Proceedings of the 29th USENIX Security Symposium. USENIX Security '20.

dblp, Accessed: 2025-08-31. dblp, https://dblp.uni-trier.de/ [online].

Dependabot, Accessed: 2025-08-31. Dependabot, https://github.com/dependabot [online].

Derr, Erik, Bugiel, Sven, Fahl, Sascha, Acar, Yasemin, Backes, Michael, 2017. Keep me updated: An empirical study of third-party library updatability on android. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. CCS '17.

Duan, Ruian, Bijlani, Ashish, Xu, Meng, Kim, Taesoo, Lee, Wenke, 2017. Identifying open-source license violation and 1-day security risk at large scale. In: Proceedings of the 24th ACM SIGSAC Conference on Computer and Communications Security. CCS '17.

Duan, Yue, Gao, Lian, Hu, Jie, Yin, Heng, 2019. Automatic Generation of Non-intrusive Updates for Third-Party Libraries in Android Applications. In: Proceedings of the 22nd International Symposium on Research in Attacks, Intrusions and Defenses. RAID '19.

Fdroid, Accessed: 2025-08-31. Fdroid, https://f-droid.org/ [online].

Github, Accessed: 2025-08-31. What is software composition analysis (SCA)?, https://github.com/resources/articles/security/what-is-software-composition-analysis#how-does-sca-work [online].

Github, Accessed: 2025-08-31. Github, https://github.com/ [online].

Gonzalez, Hugo, Stakhanova, Natalia, Ghorbani, Ali A., 2015. Droidkin: Lightweight detection of android apps similarity. In: Proceedings of the 10th International Conference on on Security and Privacy in Communication Networks. ICST '14.

Google, Accessed: 2025-08-31. shrink code optimization, https://developer.android.google.cn/studio/build/shrinkcode#optimization [online].

Google, Accessed: 2025-08-31. Android Studio, https://developer.android.google.cn/studio/ [online].

Google, Accessed: 2025-08-31. Gradle build overview, https://developer.android.com/build/gradle-build-overview [online].

Google, Accessed: 2025-08-31. Google Scholar, https://scholar.google.com/ [online].

Gradle, Accessed: 2025-08-31. Gradle build tool, https://gradle.org/ [online].

Guardsquare, Accessed: 2025-08-31. ProGuard: Java Obfuscator and Android App Optimizer, https://www.guardsquare.com/proguard [online].

Han, Hongmu, Li, Ruixuan, Tang, Junwei, 2018. Identify and inspect libraries in android applications. Wirel. Pers. Commun..

Hanna, Steve, Huang, Ling, Wu, Edward, Li, Saung, Chen, Charles, Song, Dawn, 2013. Juxtapp: A scalable system for detecting code reuse among android applications. In: Proceedings of the 9th International Conference of Detection of Intrusions and Malware, and Vulnerability Assessment.

Huang, Jie, Borges, Nataniel, Bugiel, Sven, Backes, Michael, 2019. Up-to-crash: Evaluating third-party library updatability on android. In: Proceedings of the 2019 IEEE European Symposium on Security and Privacy. EuroS&P '19.

Huang, Jianjun, Xue, Bo, Jiang, Jiasheng, You, Wei, Liang, Bin, Wu, Jingzheng, Wu, Yanjun, 2022. Scalably detecting third-party android libraries with two-stage bloom filtering. IEEE Trans. Softw. Eng..

IIE-LibD, Accessed: 2025-08-31. LibD https://github.com/IIE-LibD/libd/issues [online].

Imtiaz, Nasif, Thorn, Seaver, Williams, Laurie, 2021. A comparative study of vulnerability reporting by software composition analysis tools. In: Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement. ESEM '21.

Jarukitpipat, Vipawan, Chhun, Klinton, Wanprasert, Wachirayana, Ragkhitwetsagul, Chaiyong, Choetkiertikul, Morakot, Sunetnanta, Thanwadee, Kula, Raula Gaikovina, Chinthanet, Bodin, Ishio, Takashi, Matsumoto, Kenichi, 2023. V-Achilles: An interactive visualization of transitive security vulnerabilities. In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. ASE '22.

JesusFreke, Accessed: 2025-08-31. Baksmali, https://github.com/JesusFreke/smali [online].

Jiang, Ling, An, Junwen, Huang, Huihui, Tang, Qiyi, Nie, Sen, Wu, Shi, Zhang, Yuqun, 2024. BinaryAI: Binary software composition analysis via intelligent binary source code matching. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24.

Jiang, Ling, Yuan, Hengchen, Tang, Qiyi, Nie, Sen, Wu, Shi, Zhang, Yuqun, 2023. Third-party library dependency for large-scale SCA in the C/C++ ecosystem: How far are we? In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. ISTTA '23.

Jiang, Zheyue, Zhang, Yuan, Xu, Jun, Wen, Qi, Wang, Zhenghe, Zhang, Xiaohan, Xing, Xinyu, Yang, Min, Yang, Zhemin, 2020. PDiff: Semantic-based patch presence testing for downstream kernels. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. CCS '20.

Kim, Seulbae, Woo, Seunghoon, Lee, Heejo, Oh, Hakjoo, 2017. Vuddy: A scalable approach for vulnerable code clone discovery. In: Proceedings of the 2017 IEEE Symposium on Security and Privacy. S&P '17.

Kornblum, Jesse, 2006. Identifying almost identical files using context triggered piecewise hashing. Digit. Investig..

Lemieux, Caroline, Inala, Jeevana Priya, Lahiri, Shuvendu K, Sen, Siddhartha, 2023. Codamosa: Escaping coverage plateaus in generation with pre-trained large language models. In: Proceedings of the IEEE/ACM 45th International Conference on Software Engineering. ICSE '23.

Li, Li, Bissyandé, Tegawendé F., Klein, Jacques, Le Traon, Yves, 2016. An investigation into the use of common libraries in android apps. In: Proceedings of the 23rd International Conference on Software Analysis, Evolution, and Reengineering. SANER '16.

Li, Li, Li, Daoyuan, Bissyandé, Tegawendé F., Klein, Jacques, Cai, Haipeng, Lo, David, Le Traon, Yves, 2017a. Automatically locating malicious packages in piggybacked android apps. In: Proceedings of the 4th International Conference on Mobile Software Engineering and Systems. MOBILESoft '17.

Li, Siyuan, Wang, Yongpan, Dong, Chaopeng, Yang, Shouguo, Li, Hong, Sun, Hao, Lang, Zhe, Chen, Zuxin, Wang, Weijie, Zhu, Hongsong, et al., 2023. Libam: An area matching framework for detecting third-party libraries in binaries. ACM Trans. Softw. Eng. Methodol..

Li, Menghao, Wang, Wei, Wang, Pei, Wang, Shuai, Wu, Dinghao, Liu, Jian, Xue, Rui, Huo, Wei, 2017b. Libd: Scalable and precise third-party library detection in android markets. In: 2017 IEEE/ACM 39th International Conference on Software Engineering. ICSE '17.

Liu, Bin, Liu, Bin, Jin, Hongxia, Govindan, Ramesh, 2015. Efficient privilege de-escalation for ad libraries in mobile apps. In: Proceedings of the 13th Annual International Conference on Mobile Systems, Applications, and Services. MobiSys '15.

Ma, Ziang, Wang, Haoyu, Guo, Yao, Chen, Xiangqun, 2016. LibRadar: Fast and Accurate Detection of Third-party Libraries in Android Apps. In: Proceedings of the 38th International Conference on Software Engineering Companion. ICSE-Companion '16.

Merkle, Ralph C., 1987. A digital signature based on a conventional encryption function. In: Proceedings of the 7th International Conference on the Theory and Applications of Cryptographic Techniques. CRYPTO '87.

Ntousakis, Grigoris, Ioannidis, Sotiris, Vasilakis, Nikos, 2021. Demo: Detecting third-party library problems with combined program analysis. In: Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. CCS '21.

NVD, Accessed: 2025-08-31. CVE-2020-8840, https://nvd.nist.gov/vuln/detail/cve-2020-8840 [online].

OSSSanitizer, Accessed: 2025-08-31. OSSPolice, https://github.com/osssanitizer/osspolice [online].

OWASP, Accessed: 2025-08-31. OWASP Dependency-Check, https://owasp.org/www-project-dependency-check/ [online].

Pan, Zhiyuan, Hu, Xing, Xia, Xin, Zhan, Xian, Lo, David, Yang, Xiaohu, 2024. Ppt4J: Patch presence test for java binaries. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24.

Plate, Henrik, Ponta, Serena Elisa, Sabetta, Antonino, 2015. Impact assessment for vulnerabilities in open-source software libraries. In: Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution. ICSME '15.

Ponta, Serena Elisa, Plate, Henrik, Sabetta, Antonino, 2018. Beyond metadata: Code-centric and usage-based analysis of known vulnerabilities in open-source software. In: Proceedings of the 2018 IEEE International Conference on Software Maintenance and Evolution. ICSME '18.

Ponta, Serena Elisa, Plate, Henrik, Sabetta, Antonino, 2020. Detection, assessment and mitigation of vulnerabilities in open source dependencies. Empir. Softw. Eng..

PreEmptive, Accessed: 2025-08-31. Dasho Professional-grade Application Protection, https://www.preemptive.com/products/dasho/ [online].

pxb1988, Accessed: 2025-08-31. dex2jar, https://github.com/pxb1988/dex2jar [online].

sdhash@roussev.net, Accessed: 2025-08-31. sdhash, http://roussev.net/sdhash/sdhash.html [online].

Soh, Charlie, Tan, Hee Beng Kuan, Arnatovich, Yauhen Leanidavich, Narayanan, Annamalai, Wang, Lipo, 2016. LibSift: Automated Detection of Third-party Libraries in Android Applications. In: Proceedings of the 23rd Asia-Pacific Software Engineering Conference. APSEC '16.

Sonatype, Accessed: 2025-08-31. Ossindex, https://ossindex.sonatype.org/ [online].

Soot-oss, Accessed: 2025-08-31. Soot, https://soot-oss.github.io/soot/ [online].

Statcounter, Accessed: 2025-08-31. Mobile operating system market share worldwid, https://gs.statcounter.com/os-market-share/mobile/worldwide [online].

Synk, Accessed: 2025-08-31. Synk, https://snyk.io/ [online].

Tang, Chongbin, Chen, Sen, Fan, Lingling, Xu, Lihua, Liu, Yang, Tang, Zhushou, Dou, Liang, 2019a. A large-scale empirical study on industrial fake apps. In: Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice. ICSE-SEIP' 19.

Tang, Wei, Luo, Ping, Fu, Jialiang, Zhang, Dan, 2020. LibDX: A cross-platform and accurate system to detect third-party libraries in binary code. In: Proceedings of 27th International Conference on Software Analysis. SANER '20.

Tang, Zhushou, Xue, Minhui, Meng, Guozhu, Ying, Chengguo, Liu, Yugeng, He, Jianan, Zhu, Haojin, Liu, Yang, 2019b. Securing android applications via edge assistant third-party library detection. Comput. Secur..

The Hacker News, Accessed: 2025-08-31. Facebook SDK Vulnerability Puts Millions of Smartphone Users' Accounts at Risk, https://thehackernews.com/2014/07/facebook-sdk-vulnerability-puts.html [online].

The Hacker News, Accessed: 2025-08-31. Backdoor in Baidu Android SDK Puts 100 Million Devices at Risk, https://thehackernews.com/2015/11/android-malware-backdoor.html [online].

Tomassi, David A., Nwodo, Kenechukwu, Elsabagh, Mohamed, 2023. Libra: Library identification in obfuscated android apps. In: Proceedings of the 2023 International Conference on Information Security. Inscrypt '23.

Viertel, Fabien Patrick, Kortum, Fabian, Wagner, Leif, Schneider, Kurt, 2019. Are third-party libraries secure? A software library checker for java. In: Proceedings of the 13th International Conference on Risks and Security of Internet and Systems. CRiSIS '18.

WALA, Accessed: 2025-08-31. WALA, https://github.com/wala/WALA.

Wang, Haoyu, Guo, Yao, 2017. Understanding third-party libraries in mobile app analysis. In: Proceedings of the 39th International Conference on Software Engineering Companion. ICSE-Companion '17.

Wang, Haoyu, Guo, Yao, Ma, Ziang, Chen, Xiangqun, 2015. WuKong: A scalable and accurate two-phase approach to android app clone detection. In: Proceedings of the 2015 International Symposium on Software Testing and Analysis. ISSTA '15.

Wang, Yan, Wu, Haowei, Zhang, Hailong, Rountev, Atanas, 2018. ORLIS: obfuscation-resilient library detection for android. In: Proceedings of the 5th International Conference on Mobile Software Engineering and Systems. MOBILESoft '18.

Watanabe, Takuya, Akiyama, Mitsuaki, Kanei, Fumihiro, Shioji, Eitaro, Takata, Yuta, Sun, Bo, Ishi, Yuta, Shibahara, Toshiki, Yagi, Takeshi, Mori, Tatsuya, 2017. Understanding the origins of mobile app vulnerabilities: A large-scale measurement study of free and paid apps. In: Proceedings of 14th International Conference on Mining Software Repositories. MSR '17.

Wermke, Dominik, Huaman, Nicolas, Acar, Yasemin, Reaves, Bradley, Traynor, Patrick, Fahl, Sascha, 2018. A large scale investigation of obfuscation use in google play. In: Proceedings of the 34th Annual Computer Security Applications Conference. ACSAC '18.

Wikipedia, Accessed: 2025-08-31. Bloom filter, https://en.wikipedia.org/wiki/Bloom_filter [online].

Wikipedia, Accessed: 2025-08-31. Jaccard Index, https://en.wikipedia.org/wiki/Jaccard_index [online].

Woo, Seunghoon, Choi, Eunjin, Lee, Heejo, Oh, Hakjoo, 2023. V1SCAN: Discovering 1-day vulnerabilities in reused C/C++ open-source software components using code classification techniques. In: Proceedings of the 32nd USENIX Security Symposium. USENIX '23.

Woo, Seunghoon, Hong, Hyunji, Choi, Eunjin, Lee, Heejo, 2022. Movery: A precise approach for modified vulnerable code clone discovery from modified open-source software components. In: Proceedings of the 31st USENIX Security Symposium. USENIX '22.

Woo, Seunghoon, Lee, Dongwook, Park, Sunghan, Lee, Heejo, Dietrich, Sven, 2021a. V0Finder: Discovering the correct origin of publicly reported software vulnerabilities. In: Proceedings of the 30th USENIX Security Symposium. USENIX '21.

Woo, Seunghoon, Park, Sunghan, Kim, Seulbae, Lee, Heejo, Oh, Hakjoo, 2021b. Centris: A precise and scalable approach for identifying modified open-source software reuse. In: Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering. ICSE '21.

Wu, Yichen, Pan, Xudong, Hong, Geng, Yang, Min, 2025. OpenDeception: Benchmarking and Investigating AI Deceptive Behaviors via Open-ended Interaction Simulation. arXiv preprint arXiv:2504.13707.

Wu, Yafei, Sun, Cong, Zeng, Dongrui, Tan, Gang, Ma, Siqi, Wang, Peicheng, 2023. LibScan: Towards More Precise Third-Party Library Identification for Android Applications. In: Proceedings of the 32nd USENIX Security Symposium. USENIX '23.

Xi, Zhiheng, Chen, Wenxiang, Guo, Xin, He, Wei, Ding, Yiwen, Hong, Boyang, Zhang, Ming, Wang, Junzhe, Jin, Senjie, Zhou, Enyu, et al., 2025. The rise and potential of large language model based agents: A survey. Sci. China Inf. Sci..

Xiao, Yang, Chen, Bihuan, Yu, Chendong, Xu, Zhengzi, Yuan, Zimu, Li, Feng, Liu, Binghong, Liu, Yang, Huo, Wei, Zou, Wei, et al., 2020. MVP: Detecting vulnerabilities using Patch-Enhanced vulnerability signatures. In: Proceedings of the 29th USENIX Security Symposium. USENIX Security '20.

Xie, Zifan, Wen, Ming, Jia, Haoxiang, Guo, Xiaochen, Huang, Xiaotong, Zou, Deqing, Jin, Hai, 2023. Precise and efficient patch presence test for android applications against code obfuscation. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA '23.

Xie, Zifan, Wen, Ming, Li, Tinghan, Zhu, Yiding, Hou, Qinsheng, Jin, Hai, 2024. How does code optimization impact third-party library detection for android applications? In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. ASE '24.

Xu, Shangzhi, Dong, Jialiang, Cai, Weiting, Li, Juanru, Shaghaghi, Arash, Sun, Nan, Ma, Siqi, 2025. Enhancing security in third-party library reuse–comprehensive detection of 1-day vulnerability through code patch analysis. In: Proceedings of the 32nd Network and Distributed System Security Symposium. NDSS '25.

Xu, Yifei, Xu, Zhengzi, Chen, Bihuan, Song, Fu, Liu, Yang, Liu, Ting, 2020. Patch based vulnerability matching for binary programs. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA '20.

Xu, Jian, Yuan, Qianting, 2020. LibRoad: Rapid, Online, and Accurate Detection of TPLs on Android. IEEE Trans. Mob. Comput..

Yasumatsu, Tatsuhiko, Watanabe, Takuya, Kanei, Fumihiro, Shioji, Eitaro, Akiyama, Mitsuaki, Mori, Tatsuya, 2019. Understanding the Responsiveness of Mobile App Developers to Software Library Updates. In: Proceedings of the 9th ACM Conference on Data and Application Security and Privacy. CODASPY '19.

Yuan, Zimu, Feng, Muyue, Li, Feng, Ban, Gu, Xiao, Yang, Wang, Shiyang, Tang, Qian, Su, He, Yu, Chendong, Xu, Jiahuan, et al., 2019. B2SFinder: Detecting open-source software reuse in COTS software. In: Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering. ASE '19.

Zhan, Xian, Fan, Lingling, Chen, Sen, We, Feng, Liu, Tianming, Luo, Xiapu, Liu, Yang, 2021a. Atvhunter: Reliable version detection of third-party libraries for vulnerability identification in android applications. In: 2021 IEEE/ACM 43rd International Conference on Software Engineering. ICSE '21.

Zhan, Xian, Fan, Lingling, Liu, Tianming, Chen, Sen, Li, Li, Wang, Haoyu, Xu, Yifei, Luo, Xiapu, Liu, Yang, 2020. Automated third-party library detection for android applications: Are we there yet? In: Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering. ASE '20.

Zhan, Qi, Hu, Xing, Li, Zhiyang, Xia, Xin, Lo, David, Li, Shanping, 2024a. PS^3: Precise Patch Presence Test based on Semantic Symbolic Signature. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. ICSE '24.

Zhan, Qi, Hu, Xing, Xia, Xin, Li, Shanping, 2024b. REACT: IR-Level Patch Presence Test for Binary. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. ASE '24.

Zhan, Xian, Liu, Tianming, Fan, Lingling, Li, Li, Chen, Sen, Luo, Xiapu, Liu, Yang, 2021b. Research on third-party libraries in android apps: A taxonomy and systematic literature review. IEEE Trans. Softw. Eng..

Zhan, Xian, Liu, Tianming, Liu, Yepang, Liu, Yang, Li, Li, Wang, Haoyu, Luo, Xiapu, 2021c. A systematic assessment on android third-party library detection tools. IEEE Trans. Softw. Eng..

Zhan, Xian, Zhang, Tao, Tang, Yutian, 2019. A comparative study of android repackaged apps detection techniques. In: Proceedings of the 26th International Conference on Software Analysis, Evolution and Reengineering. SANER '19.

Zhang, Jiexin, Beresford, Alastair R., Kollmann, Stephan A., 2019. LibID: Reliable identification of obfuscated third-party android libraries. In: Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis. ISSTA '19.

Zhang, Yuan, Dai, Jiarun, Zhang, Xiaohan, Huang, Sirong, Yang, Zhemin, Yang, Min, Chen, Hao, 2018. Detecting third-party libraries in android applications with high precision and recall. In: Proceedings of the 25th International Conference on Software Analysis, Evolution and Reengineering. SANER '18.

Zhang, Zicheng, Diao, Wenrui, Hu, Chengyu, Guo, Shanqing, Zuo, Chaoshun, Li, Li, 2020. An empirical study of potentially malicious third-party libraries in android apps. In: Proceedings of the 13th ACM Conference on Security and Privacy in Wireless and Mobile Networks. WiSec '20.

Zhang, Hang, Qian, Zhiyun, 2018. Precise and accurate patch presence test for binaries. In: Proceedings of the 27th USENIX Security Symposium. USENIX Security '18.

Zhao, Lida, Chen, Sen, Xu, Zhengzi, Liu, Chengwei, Zhang, Lyuye, Wu, Jiahui, Sun, Jun, Liu, Yang, 2023a. Software composition analysis for vulnerability detection: An empirical study on Java projects. In: Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. ESEC/FSE '23.

Zhao, Kaifa, Zhan, Xian, Yu, Le, Zhou, Shiyao, Zhou, Hao, Luo, Xiapu, Wang, Haoyu, Liu, Yepang, 2023b. Demystifying privacy policy of third-party libraries in mobile apps. In: Proceedings of the 45th International Conference on Software Engineering. ICSE '23.

Zhao, Wayne Xin, Zhou, Kun, Li, Junyi, Tang, Tianyi, Wang, Xiaolei, Hou, Yupeng, Min, Yingqian, Zhang, Beichen, Zhang, Junjie, Dong, Zican, et al., 2023c. A survey of large language models. arXiv preprint arXiv:2303.18223.

Zhou, Wu, Zhou, Yajin, Grace, Michael, Jiang, Xuxian, Zou, Shihong, 2013. Fast, scalable detection of "piggybacked" mobile applications. In: Proceedings of the Third ACM Conference on Data and Application Security and Privacy.

Zhou, Wu, Zhou, Yajin, Jiang, Xuxian, Ning, Peng, 2012. Detecting repackaged smartphone applications in third-party android marketplaces. In: Proceedings of the Second ACM Conference on Data and Application Security and Privacy. CODASPY '12.